АНОТАЦІЯ

Дана курсова робота присвячена розробці програмної системи для моделювання битви добра та зла – героїв гри Super Mario, з використанням UML та мови програмування Java та об’єктно-орієнтованих принципів. Програма включає в себе роботу з файлами, класами, обробку числової інформації і роботу з графікою. В ході роботи було створено програмну оболонку поставленої задачі і розроблено методи графічного моделювання поведінки героїв гри Suoer Mario. В результаті виконання курсової роботи отримано програмний засіб, працездатність і правильність роботи якого перевірено, підготовлена інструкція користувача.

ЗМІСТ

ВСТУП……...5

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ……..…..8

2 РОЗРОБКА ІНТЕРФЕЙСУ ПРОГРАМИ ТА ПРИКЛАДІВ ВИКОРИСТАННЯ……17

3 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ….21

4 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ’ЄКТІВ ТА СТАНУ……23

5 ВИКОРИСТАННЯ ЗАСОБІВ ПРОГРАМУВАННЯ JAVAFX…....28

6 РОЗРОБКА ПІДСИСТЕМИ СЕРІАЛІЗАЦІЇ/ДЕСЕРІАЛІЗАЦІЇ ДАНИХ……30

7 КЕРІВНИЦТВО КОРИСТУВАЧА……..…….32

ВИСНОВКИ…….……..35

ПЕРЕЛІК ПОСИЛАНЬ.……36

Додаток А. Лістинг модуля Main.java.. ……..…38

Додаток Б . Лістинг модуля Mario.java……..…….42

Додаток В. Лістинг модуля Princess.java…….48

Додаток Г. Лістинг модуля Speed.java…….……...……50

Додаток Д. Лістинг модуля Pilz.java……52

Додаток Е. Лістинг модуля Wallpaper.java.. ……..……54

Додаток Ж. Лістинг модуля TopMenu.java……..55

Додаток И. Лістинг модуля BadBase.java…….58

Додаток К. Лістинг модуля GoodBase.java……..59

Додаток Л. Лістинг модуля Dial.java……...……60

Додаток М. Лістинг модуля Drago.java ……62

ВСТУП

Персонаж Маріо вперше з'явився в грі для ігрового автомата Donkey Kong у 1981 році [1] де він був головним героєм, носив ім'я Джампмен (англ. Jumpman, в буквальному перекладі Стрибун) і був теслярем. Згодом ця гра отримала ряд продовжень і стала окремою серією ігор. Першою грою з Маріо в якості головного героя стала Mario Bros(в якій в Mario нова професія—сантехнік). Також в грі вперше з'являються такі персонажі, як брат Маріо — Луїджі (англ. Luigi) і черепахи Купа (англ. Koopa).

Гравці контролювали Маріо, або його брата Луїджі в багатокористувацькому режимі, вони подорожують Грибним Королівством, щоб врятувати принцесу від антагоніста Баузера (англ. Bowser). Вони повинні були уникати небезпек, таких як вороги і ями, не без допомоги бонусів (Super Mushroom, Fire Flower та Starman). Гра була розроблена Шигерою Міямото і Такаші Тезукою. Музику написав Кодзі Кондо.

Остаточно концепція ігор серії сформувалася в грі Super Mario Bros., яка побачила світ наприкінці 1985 року ексклюзивно для ігрової консолі Nintendo Entertainment System. Гра стала платформером з горизонтальною прокруткою екрана. Представлено Грибне королівство – всесвіт в якому розгортається дія гри, вперше з'явилися деякі противники і персонажі, такі як Принцеса Піч і Тоад. У грі було вісім рівнів-світів, кожен з яких складався з чотирьох підрівнів. Герой міг розбивати цегляні блоки, отримувати призи, що вибивав зі спеціальних блоків зі знаком питання, а також—збирати монети, атакувати ворогів за допомогою стрибка на них зверху. Також з'явився сюжет, який було згодом використано в іграх серії—герой повинен був врятувати викрадену принцесу і звільнити Грибне Королівство, що було захоплене черепахами Купа. Кожен з світів закінчувався зоною в замку, в якому знаходилася викрадена принцеса, але після перемоги над босом герой зустрічав Тоада, який повідомляв гравцеві, що принцеса знаходиться в іншому замку.

Super Mario Bros. часто цитується як одна з найкращих відеоігор усіх часів. Було продано понад 40 мільйонів фізичних копій, що також робить її однією з бестселерів усіх часів. Super Mario разом з NES були одним із ключових факторів у відродженні індустрії відеоігор після занепаду 1983 року, і допомогли популяризувати платформні ігри. Порти і ремейки гри були випущені для більшості майбутніх систем Nintendo. Поряд з самим Маріо, Super Mario Bros. став важливим аспектом попкультури.

Згодом в 1988 році вийшла Super Mario Bros 2. розроблена і видана Nintendo для консолі NES як продовження Super Mario Bros. Гра представляла помітно змінений геймплей. Основна зміна торкнулося схеми взаємодії з супротивниками: прийнята в попередніх іграх серії модель, в якій Маріо міг вбити ворога, стрибнувши на нього зверху (якщо тільки це не був дикобраз або хто-небудь інший, захищений шипами) була усунена. Замість цього була додана можливість кидати в супротивника різні предмети, розподілені з ігрових рівнів (найчастіше висмикнуті з землі овочі, а також інших противників). Предмет можна взяти, попередньо стрибнувши йому на голову. Таким чином у Super Mario Bros. 2 супротивника можна вбити тільки за допомогою стороннього предмета, іншого супротивника, або взявши бонус — «зірку», що дає невразливість. У грі був введений рівень життя, вимірюваний кількістю червоних шестикутниківу лівому верхньому куті екрану. У попередніх іграх величини «здоров'я» не було, при контакті з супротивником Маріо моментально втрачав життя, а якщо в цей момент він перебував у «просунутому» стані, то повертався до попереднього (Fiery Mario і Super Mario ставали звичайним Mario). Таким чином у Super Mario Bros. 2 гравець міг кілька разів зіткнутися з противником, перш ніж втратити одне життя. Також гравцеві була надана можливість вибору одного з чотирьох ігрових персонажів: Маріо, Луїджі, Піч і Тоад. Раніше гравець міг вибирати тільки між Маріо і Луїджі (і лише в режимі двох гравців), при цьому різниця між персонажами полягало в тому, що Луїджі стрибав далі Маріо і сильніше «пробуксовував». У той же час всі чотири персонажі Super Mario Bros. 2

володіють унікальними для кожного з них здібностями, визначальними стиль проходження гри. Крім цього в гру була додана міні-гра, що працює за принципом однорукого бандита, в якій гравець може заробити бонуси, що полегшують проходження.

Super Mario Bros. і більшість наступних ігор серії виходили ексклюзивно для ігрових консолей Nintendo. В країнах СНД дані ігри можна було пограти на Dendy – клоні ігрової консолі Nintendo.

Всі ігри франшизи створені виключно для ігрової приставки Nintendo. На жаль жодну з них не вийде пограти на улюбленій платформі, як от Windows.

Їм очевидно бракує таких речей як: можливості грати на будь-якій платформі(Windows, MacOS, Linux, Android), невибагливості до апаратної частини комп’ютера. Саме новітні засоби JavaFX здатні вирішити ці проблеми і навіть більше – вдосконалити графічну частину.

Тому, було поставлено за мету надати користувачам ПК можливість абсолютно безкоштовно пограти в власну реалізацію Super Mario, при цьому без необхіності купувати дорогу приставку і ще й окремо до неї досить не дешеві ігри.

1 АНАЛІЗ СУЧАСНОГО СТАНУ ПИТАННЯ ТА ОБҐРУНТУВАННЯ ЗАВДАННЯ НА РОБОТУ

1.1 Предметна область

У Маріо є три відмітних ознаки—вуса, кепка і комбінезон. Вуса у Маріо, щоб візуально відокремити ніс від лиця, кепка—тому що на спрайт маленької роздільної здатності було неможливо намалювати зачіску, комбінезон—щоб краще передати рухи рук.

Динозавр Йоші є одним з найвідоміших героїв Грибного королівства. Його батьківщина — острів, на якому живуть інші динозаври Йоші (англ. Yoshi's Island). Неодноразово допомагав героям ігор Маріо в їх боротьбі з Боузером і іншими злодіями. Доглядав Маріо і Луїджі(брата Маріо), коли ті були немовлятами, і захищав їх від злого чаклуна Купи і його поплічників. Як і інші динозаври Йоші, витривалий(має багато життєвих сил та швидше їх відновлює), також він бігає швидше ніж Маріо. Варто зазначити, що надалі іменами Йоші(Yoshi) та Драго(Drago) буде називатись один і той самий динозавр з гри Super Mario.

Принцеса Піч найчастіше зображується одягненою в рожеву сукню, з золотистим волоссям і витонченою короною на голові. У неї добра натура і хороші манери, в більшості ігор Піч зображується безкорисливою і великодушною, не виявляє агресивності в жодній формі, навіть при зіткненні зі своїми ворогами. І хоча в деяких іграх лиходії все ж провокують у неї гнів, переважно Піч веде себе благопристойно і стримано. Хоч вона і в цій реалізації гри зображується крихкою та тендітною(має мало життєвих сил) та є набагато сильнішою ніж Маріо чи Йоші, адже саме в її руках сила всього Грибного Королівства.

Гриби – макрооб’єкти. Герої(мікрооб’єкти) намагаються випередити антагоністів (антигероїв) й врятувати Грибне Королівство від їхньої навали, захоплюючи якомога більше грибів. Колір грибів змінюється в залежності від того, яка сила їх захопила. Незахоплені гриби жовтого кольору, якщо їх захопили

Герої, то вони змінюють свій колір на білий, якщо ж Антигерої, то на чорний. Гриби розміщені в випадкових точках карти, їх кількість є обмеженою. Всі персонажі - як герої, так і антигерої можуть захоплювати гриби. Маріо та Йоші – захисники Принцеси, і відповідно АнтиМаріо та АнтиЙоші – захисники АнтиПринцеси. АнтиГерої або Поганці(англ. Bad) та Герої або Добряки (англ. Good) мають відмінну зовнішню ознаку – літеру “B” та “G” на своєму вбранні відповідно. Дві сторони б’ються між собою, “дружні” гриби, тобто гриби захоплені союзниками відновлюють їм здоров'я якщо ті знаходяться на грибу, якщо ж на гриб наступає противник, то в нього суттєво зменшуються життєві сили. В гри є табло, якщо кількість незахоплених грибів вичерпана, то перемагає та зі сторін, яка захопила найбільше грибів.

Основною проблемою є вирішення конфлікту, конфлікту королівств, в метафізичному сенсі – конфлікту особистостей. Боротьба добра зі злом, темряви зі сітлом…

Потрібно розуміти, що не завжди виходитиме 0 чи 1. Це може бути якесь дробове, чи ірраціональне, чи взагалі комплексне число – таке життя, воно повне злетів і падіннь. Тому покликання цієї гри вкотре показати і допомогти користувачеві вирішити будь-який конфлікт шляхом простого зведення конфлікту до протистояння добра та зла.

1.2 Існуючі реалізації

Рисунок 1.1 – Фрагмент гри Super Mario Bros

На сьогодні існує ціла серія ігор про Mario: Super Mario Bros.; Super Mario Bros. 2; Super Mario Land; Super Mario Bros. 3; Super Mario World; Super Mario Galaxy; New Super Mario Bros. Wii.

Super Mario Bros (див. рисунок 1.1) – перша із ігор серії яка побачила світ на початку 1986 року та є однією із найвідоміших ігор серії. Випускалася для ігрових автоматів та консолей NES.

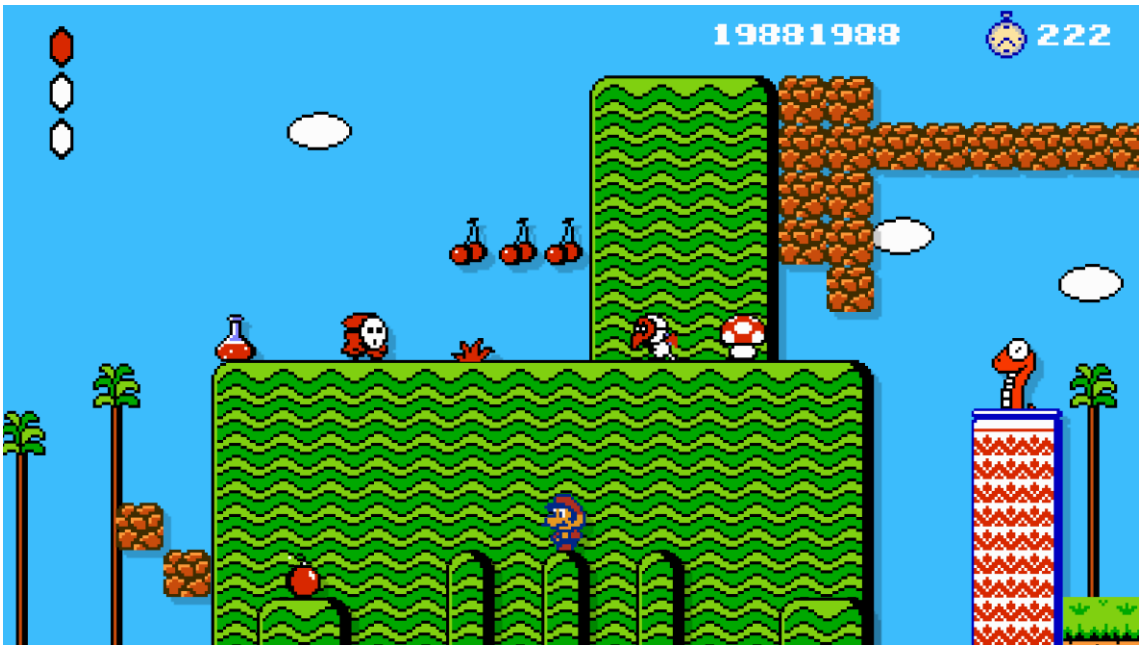


Рисунок 1.2 – Фрагмент гри Super Mario Bros 2

Super Mario Bros 2 (див. рисунок 1.2) — відеогра в жанрі платформної аркади, розроблена і видана Nintendo для консолі NES, продовження Super Mario Bros. У Північній Америці поступила в продаж в жовтні 1988 року, в Європі — у квітні 1989-го. Являла собою перероблену версію гри Doki Doki Panic, випущеної виключно на території Японії. Super Mario Bros. 2 в Японії була видана лише в липні 1992 року під назвою Super Mario USA. Випускалася на картриджах для консолей NES, PlayChoice-10, Super NES, Game Boy Advance.

Рисунок 1.3 – Фрагмент гри Super Mario Bros 3

Super Mario Bros 3 була випущена в 1991 році. Вона додала багато нововведень в серію Super Mario Bros.(див. рисунок 1.3): додатковий екран карти, мініігри, безліч нових здібностей, ворогів і типів рівнів.

У Super Mario Bros. 3 Маріо і Луїджі рятують сім королівств семи світів, які межують з грибним королівством.

Розглянуті аналоги є недосконалими та мають застарілу графіку. Також вони були розробленими виключно для ігрових консолей NES та випускались на спеціальних картриджах. Тому було прийнято рішення розробити власну реалізацію яка, завдяки новітнім засобам мови Java, надасть користувачеві можливіть пограти в гру Mario Game, отримати виключне задоволення від її проходження завдяки новітній графіці.

1.3 Розробка технічного завдання на роботу

В програмі повинно бути реалізовано: додаток Windows з графічним інтерфейсом.

Клас повинен містити не менше 4 елементів змінних і не менше 4 методів . Як мінімум одна змінна повинна бути типу int, одна – типу double і як мінімум одна – рядок (будь якої реалізації). В класі мікрооб’єкта повинен бути присутній конструктор по замовчуванню. Додати до класу мікрооб’єкта в якості елемента посилання на невеличкий службовий об’єкт, щоб зробити необхідним глибинне копіювання. Реалізовано глибинне копіювання шляхом реалізації інтерфейсу Cloneable. Повинне бути продемонстроване використання глибинного копіювання.Продемонструвати використання як мінімум двох функцій-статичних елементів класу java.util.Arrays. Продемонструвати використання інтерфейсу Comparable або Comparer. Мікрооб'єкт: мінімум три графічних примітива (один з яких - текст). Більший макрооб'єкт: мінімум три графічних примітива (один з яких - текст). Менший макрооб'єкт: мінімум три графічних примітива(один з яких - текст). При натискуванні лівої кнопки миші на мікрооб’єкті він повинен ставати

активним/неактивним. В програмі повинна бути реалізована активація декількох об’єктів одночасно. При натискувань клавіш-стрілок активні об’єкти/об’єкт повинні рухатись у вікні програми. При натискувань клавіши Delete активні об’єкти повинні знищуватись. Якщо активного об’єкта нема – клавіша Delete ігнорується. Реалізувати в проекті: Клавіша Esc повинна відміняти активацію обєкта. Повина бути побудована ієрархія класів мікрооб'єктів, яка містить як мінімум три рівня наслідування, які можуть інстанціонуватись і зображатись на екрані. Зображення кожного рівня має відрізнятись від всіх інших хоча б одним графічним примітивом. При натискуванні клавіши Insert повинно з’являтись діалогове вікно, яке повинно визначати параметри створюваного мікрооб’єкта. В нього повинна бути додана можливість обирати до якого з класів нащадків у ієрархії наслідування він належить. Крім керуючого елемента Button у діалоговому вікні також повинні бути використані як мінімум три з наступного списку: TextBox, CheckBox, {ListBox, ChoiceBox}, RadioButton. Повинно бути продемонстровано використання динамічного поліморфізму. Додати у проект клас/класи більшого макрооб’єкта. Більший макрооб’єкт призначений містити всі мікрооб’єкти та всі менші макрооб’єкти. У вікні програми відображається лише певна частина більшого макрооб’єкта, яка не повинна перевищувати 25% його загального розміру. Мінімальний розмір більшого макрооб’єкта 1600 позицій для мікрооб’єкта (тобто на екрані одночасно можна побачити не більше квадрата 20*20 позицій для мікрооб’єкта). Реалізувати можливість змінювати частину більшого макрооб’єкта, яку спостерігає користувач (будь яким способом). Менший макрооб’єкт повинен мати можливість містити більше одного мікрооб’єкта. Не обов’язково мати можливості додавати/видаляти менші макрооб’єкти в програму. Цілком достатньо, якщо програма буде мати деяку кількість менших макрооб’єктів сгенеровану при запуску програми і яка буде незмінною протягом роботи програми. Менші макрооб’єкти повинні мати певні позиції в більшому макрооб’єкті і повинні відображатись на екраині. Позиції менших макрооб’єктів в середині більшого макрооб’єкта можуть залишатись незмінними протягом роботи програми. В більшому макрооб’єкті повинно

міститись більше одного екземпляра меншого макрооб’єкта. Мікрооб’єкти можуть або належати одному (або більшій кількості) меншому макрооб’єкту, або не належати жодному. Повинна обов'язково бути релізовано можливість вилучити мікрооб'єкт з всіх менших макрооб'єктів, в результаті чого він не буде належати жодному меншому макрооб'єкту. Всі мікрооб'єкти та менші макроб'єкти повинні належати більшому макрооб'єкту і не повинні мати змоги вийти за його межі. Повинен бути реалізований автоматичний рух деяких мікрооб’єктів: 1) при русі деякі мікрооб'єкти взаємодіють між собою (це має бути помітно візуально в тому сенсі, що або щось змінюється у їх зображенні або у характері їх руху); 2) деякі мікрооб'єкти заходять в менші макрооб'єкти та деякі виходять; 3) характер руху мікрооб'єктів повинен змінюватись при натискуванні певної клавіші клавіатури або при виборі команди меню. Під характером руху мається на увазі не просто траєкторія, а те, в який спосіб вони взаємодіють з меншими макрооб'єктами та іншими мікрооб'єктами. Повинні бути продемонстровано використання віртуальних функцій. Повинні бути продемонстровано типу доступу protected. Повинна бути запрограмована серіалізація/де-серіалізація всіх об’єктів у текстовий файл. Серіалізація/де-серіалізація обов’язково повинна зберігати не тільки власне інформацію про стан макро- та мікро-об’єктів, але й про їх позицію на екрані. При серіалізації/де-серіалізації обов’язково повинні використовуватись діалогові вікна, щоб запитати у користувача ім’я файлу та його розташування на диску (наприклад можна використовувати системне діалогове вікно FileChooser).

Для нормального виконання програми необхідно наступне апаратне забезпечення:

1. комп’ютер серії IBM PC з частотою 1 ГГц і вище;
2. 512МБ оперативної пам’яті;
3. графічний адаптер SVGA (Super Video Graphic Adapter);
4. відеокарта об’ємом пам’яті не менше 128МБ;
5. миша, клавіатура, OS Windows XP/7/8/10, GNU/Linux, OSX.
6. розмір дискового простору, що займає програма: 9,42 МБайт.

7. розмір оперативної пам’яті, що займає програма: 2 500 КБайт.

1.4 Обґрунтування вибору мови програмування

Java заслужено вважається однією з кращих мов програмування багатоцільового застосування, саме через це та переваги, вказані нижче мову Java було обрано мовою програмування проекту.

На офіційному сайті компанії TIOBE [2], що спеціалізується на контролі якості програмного забезпечення, з'явився оновлений рейтинг мов програмування за даними на 10 квітня 2019 року (рисунок 1.4 та рисунок 1.5).

Jun 2019	Jun 2018	Change	Programming Language	Ratings	Change
1	1		Java	15.004%	-0.36%
2	2		C	13.300%	-1.64%
3	4	^	Python	8.530%	+2.77%
4	3	v	C++	7.384%	-0.95%
5	6	^	Visual Basic .NET	4.624%	+0.86%
6	5	v	C#	4.483%	+0.17%

Рисунок 1.4 – Рейтинг мов програмування за популярністю

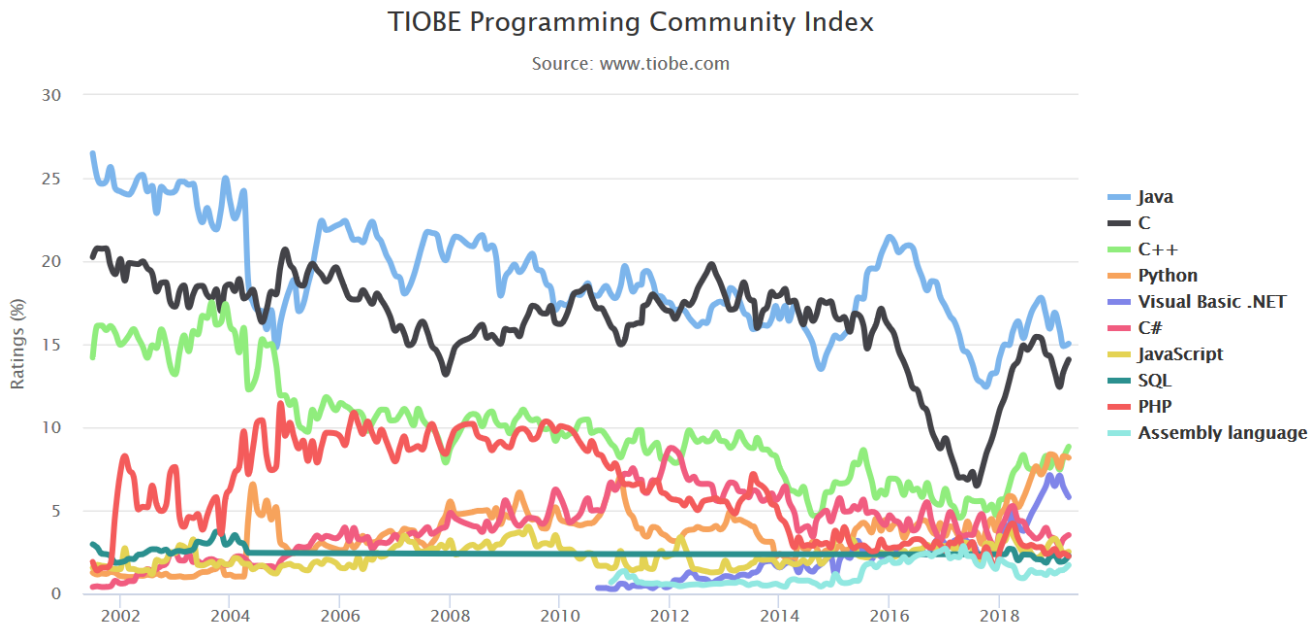


Рисунок 1.5— Зміна популярності мов програмування

Багатопотокова компіляція—можливість компілятора на багатопроцесорних системах використовувати декілька потоків для прискорення компіляції. Підтримується в Java та C++ [3].

Незалежність від платформи: C++ компілюється під кожну платформу окремо. C# підтримують не всі платформи [4]. Програма, написана на мові Java, працюватиме на будь-якій підтримуваній апаратній чи системній платформі без змін у початковому коді та перекомпіляції. Програму можна виконати на будь-якій платформі, що має встановлену віртуальну машину Java, яка інтерпретує байткод у код, пристосований до специфіки конкретної операційної системи і процесора. Це є великим плюсом саме на користь Java[5].

Динамічні масиви—наявність вбудованих в мову динамічних масивів (здатних змінювати свій розмір під час виконання програми). Динамічні масиви в Java створюються за допомогою java.util.Vector (в стандартній бібліотеці), а також за допомогою java.util.ArrayList. Java надає широкий арсенал методів(функцій) для роботи з масивами [6].

В Java реалізоване багаторівневе та ієрархічне успадкування класів, а також реалізоване множинне та гібридне спадкоємство інтерфейсів, що дозволяє досить гнучко конструювати необхідну ієрархію в програмному продукті.

1.5 Висновки

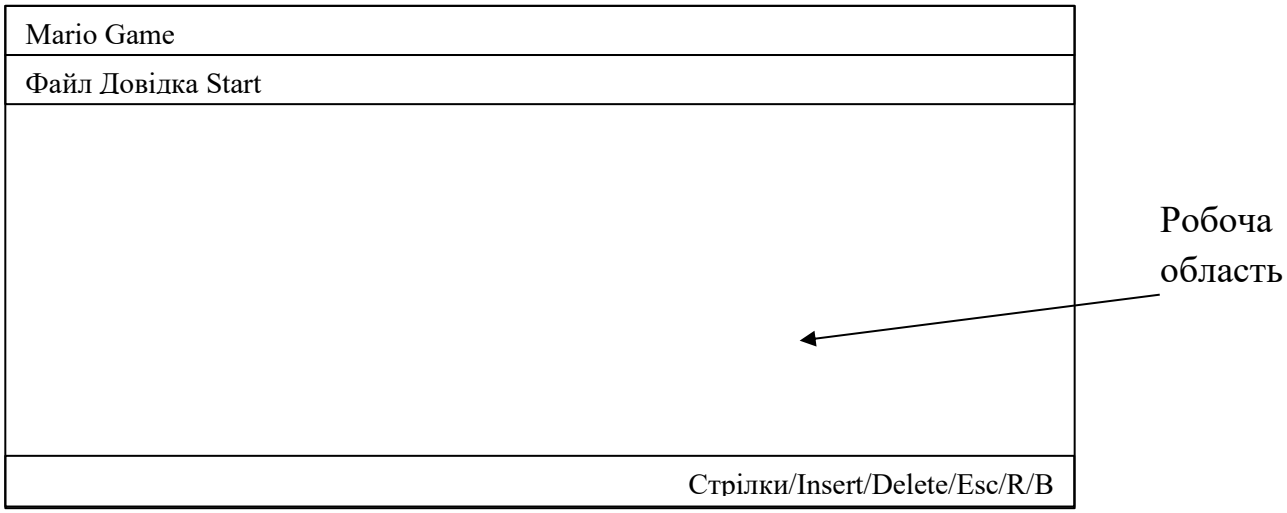
Проаналізувавши стан платформера Super Mario Bros, переглянувши усі проблеми цього питання та зваживши усі “плюси” та “мінуси” було вирішено створити власну реалізацію програмного продукту. Переглянувши існуючі реалізації схожих продуктів було підтверджено що їм не вистачає кількох важливих факторів.

Створено список завдань, які повинні бути реалізовані в програмному продукті. Найефективнішою мовою програмування для даного продукту шляхом аналізу вибрано Java.

2 РОЗРОБКА ІНТЕРФЕЙСУ ПРОГРАМИ ТА ПРИКЛАДІВ ВИКОРИСТАННЯ

2.1 Розробка інтерфейсу програми

Клієнт відображає на екрані стан внутрішнього світу і переходить в режим очікування команд від користувача (рисунок 2.1). При цьому, однак, паралельно



відбувається обробка сигналів з таймеру.

Рисунок 2.1 – Структура вікна програми

Назва вікна – «Mario Game» відповідає назві програми. Меню містить три кнопки: «Файл», «Game» та «Довідка». Робоча область вікна програми слугує для відображення об’єктів (мікро- та макрооб’єктів): героїв, грибів та мапи(великого макрооб’єкту на якому розгортаються дії). Рядок стану, стрічки прокрутки у вікні програми відсутні. Розміри вікна можна змінювати, вікно можна згорнути/розгорнути. При відкритті розмір віка сталий (600x700 px). У вкладці «Файл» містяться три команди: «Open…» (при натисканні з’являється вікно, що дає можливість відкрити файл, у якому збережена інформація про стан об’єктів), «Save…» (при натисканні з’являється вікно, що дає можливість створити файл, у якому буде збережена інформація про поточний стан об’єктів), «Exit» (при

натискуванні програма закривається). У вкладці «Довідка» натискуванням на пункт «Про програму … » можна відкрити вікно з відповідною назвою, у якому містяться відомості про продукт та розробників. У вкладці «Довідка» натискуванням на пункт «Керування … » можна відкрити вікно з відповідною назвою, у якому знаходиться опис всіх клавіш, що застосовуються для керування програмою. У вікні програми після її відкриття буде зображено гриби та дві бази гравців(добрих та злих) з випадково заданими координатами. 10 героїв з випадковою стороною(добрі або злі) з'являються у вікні. Користувач може додавати нових героїв(як добрих так і злих), змінювати їх рух, видаляти їх. Для цього використовуються команди описані далі. За замовчуванням герої одразу ж починають свій рух до найближчого макрооб'єкта(Гриба). Для користувацького руху героїв стрілками, їх треба виділити. Для виділення героя слід натиснути на нього лівою кнопкою миші. У виділеного героя зміниться колір аури(круга навколо зображення героя). Щоб зняти виділення повторно натисніть на героя або ж, щоб зняти виділення з усіх виділених героїв натисніть "ESC". Виділених героїв можна видаляти клавішею «Delete», а також клонувати за допомогою вкладки меню "Clone". Для створення нового героя викликається діалогове вікно натисненням клавіші «Insert». В ньому можна обрати клас героя, вибрати його сторону та задати швидкість руху.

2.2 Приклади використання

Приклади використання—це важливий елемент проектування. Приклад використання розкриває можливі способи взаємодії між користувачами програми та самою програмою і дозволяє розробникам приділити більше уваги покращенню зручності програми. Розглянемо декілька прикладів використання програми.

Приклад 1. Створення нового героя у діалоговому вікні (рисунок 2.2).

Натискуванням клавіші Insert з'являєтся діалогове вікно, у якому користувач має задати координати, швидкість, рівень, сторону та виділений чи ні

герой. Після натиснення кнопки «Submit» герой з’явится у відповідному місці на екрані.

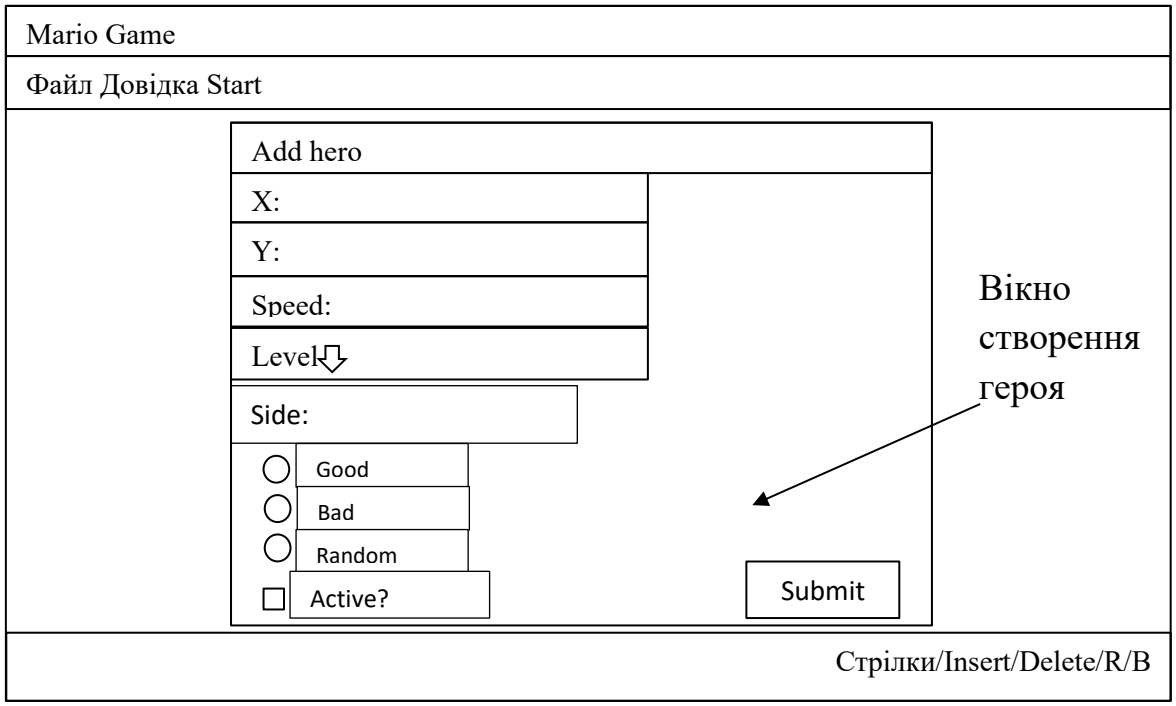
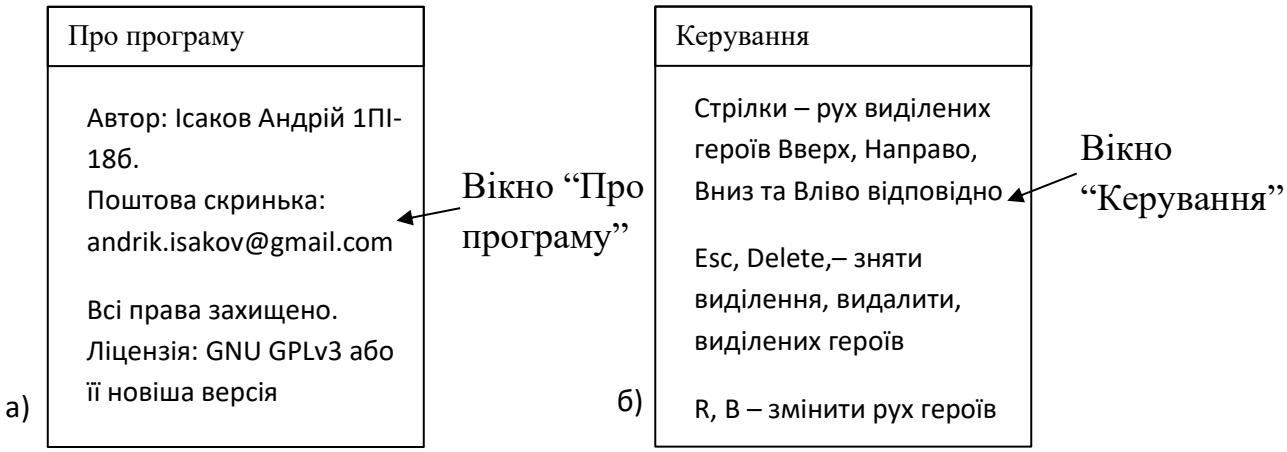


Рисунок 2.2 – Створення героя за допомогою діалогового вікна

Приклад 2. Отримання довідки(рисунок 2.3 а,б).

При натискуванні в панелі меню кнопки «Довідка» на пункт «Про програму… » відкривається вікно з відповідною назвою, у якому містяться відомості про продукт та розробників. Натискуванням на пункт «Керування … » відкривається вікно з відповідною назвою, у якому знаходиться опис всіх клавіш,

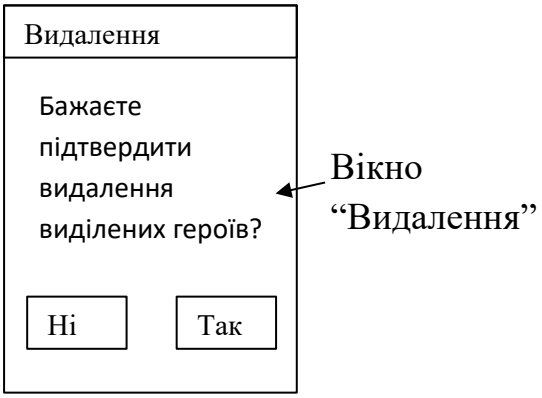


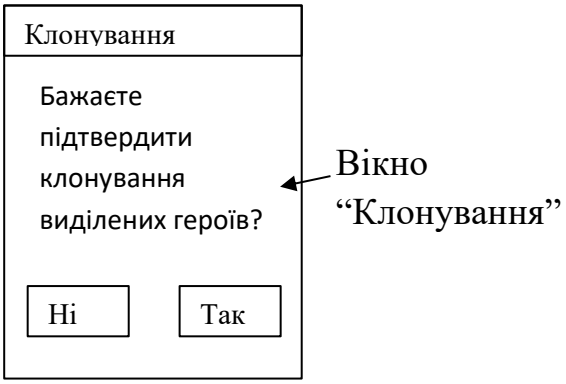
що застосовуються для керування програмою.

Рисунок 2.3 – Отримання довідки. а) – Вікно “Про програму”. б) – Вікно “Керування”

Приклад 3. Видалення героїв(рисунок 2.4)

Якщо виділено декілька героїв (декілька героїв є активними), то при натискуванні клавіші Delete, відкриється діалогове вікно підтвердження





видалення.

Рисунок 2.4 - Видалення героїв

Рисунок 2.5 - Клонування героїв

Приклад 4. Клонування героїв(рисунок 2.5)

Якщо виділено декілька героїв (декілька героїв є активними), то при натискуванні клавіші C, відкриється діалогове вікно підтвердження клонування.

3 РОЗРОБКА ПІДСИСТЕМИ ГРАФІЧНОГО ВІДОБРАЖЕННЯ

3.1 Модель графічного відображення

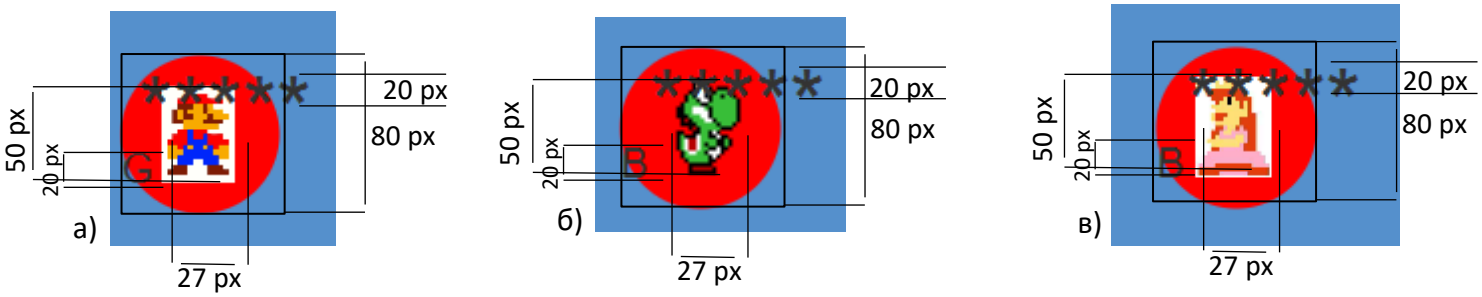
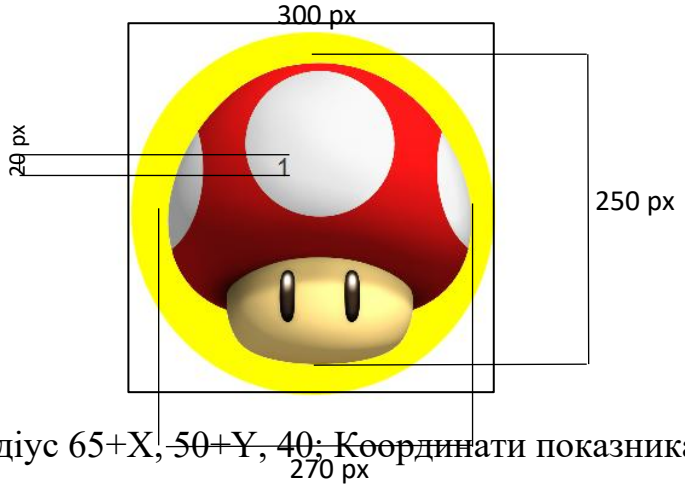


Рисунок 3.1 – Модель графічного відображення героїв а)Маріо, б)Йоші та в)Принцеси відповідно

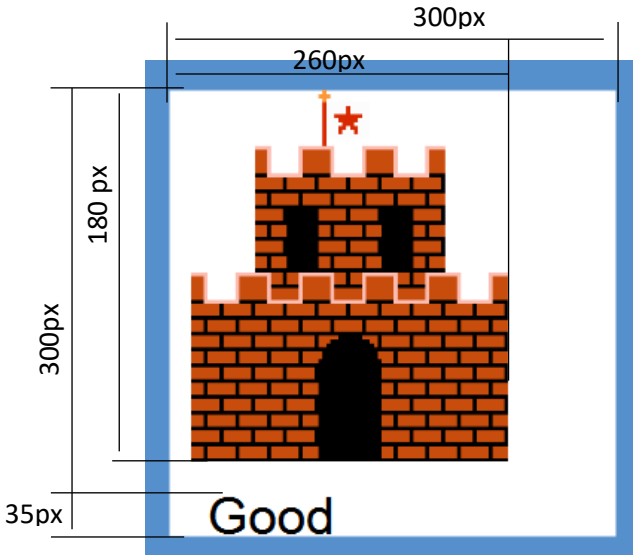
Маріо (Рисунок 3.1, а) є найслабшим героєм (1-го рівня) в ієрархії. Йоші (Рисунок 3.1, б) є героєм 2-го рівня в ієрархії героїв. Принцеса (Рисунок 3.1, в) є найсильнішим героєм (3-го рівня) в ієрархії героїв. Всі координтати, що створюють співвідношення між елементами героя (зображенням, кругом, текстом та показником здоров'я(текстом)) однакові для всіх героїв. Складається з зображення(50x27 px), круга, тексту та показнику здоров'я(тексту). Координати зображення 50+X, 25+Y; Координати тексту 40+X, 45+Y; Координати центра



круга та його радіус 65+X, 50+Y, 40; Координати показника здоров'я – 8+X, Y.

Рисунок 3.2 – Модель графічного відображення Гриба(Макрооб'єкта)

Гриб (Рисунок 3.2) є малим макрооб'єктом. Складається з зображення(250x270 px), круга та тексту. Координати зображення 50+X, 25+Y; Координати тексту 140+X, 100+Y; Координати центра круга та його радіус



170+X, 150+Y, 150.

Рисунок 3.3 – Модель графічного відображення Бази героїв

База (Рисунок 3.3) є малим допоміжним макрооб'єктом. Складається з зображення(250x270 px), прямокутника та тексту. Координати зображення 50+X, 25+Y; Координати тексту 60+X, 290+Y; Координати прямокутника та його висота й довжина відповідно: x+35, y+25, 300, 300.

3.2 Графічні процедури підсистеми графічного відображення

javafx.scene.image.ImageView - відображає картинку завантажену за допомогою Image в додатку JavaFX. javafx.scene.image.Image - представляє графічні зображення і використовується для завантаження зображень із зазначеного URL. javafx.scene.text – відображає примітивний тип String. Може задавати координати, шрифт, колір, розмір та додавати інші необхідні ефекти. javafx.scene.shape.Circle – відображає коло з заданими координатами центра та радіусом. Може задавати колір й ефекти. При створенні, коло “залите”кольором, тобто є кругом, але його можна зробити прозорим залишивши тільки коло(обведення) за допомогою дерективи setFill(Color.TRANSPARENT). javafx.scene.control.Label - використовується для відображення короткого тексту

або зображення, його не можна редагувати та він не може отримувати фокус.

4 РОЗРОБКА ДІАГРАМ КЛАСІВ, ОБ'ЄКТІВ ТА СТАНУ

4.1 Діаграми класів

На діаграмах класів показано різноманітні класи, які утворюють систему і їх взаємозв'язки[7]. На діаграмах класів показано класи разом з методами і атрибутами та статичний взаємозв'язок між ними, але не показано методи, які при цьому викликаються.

4.1.1 Діаграми наслідування

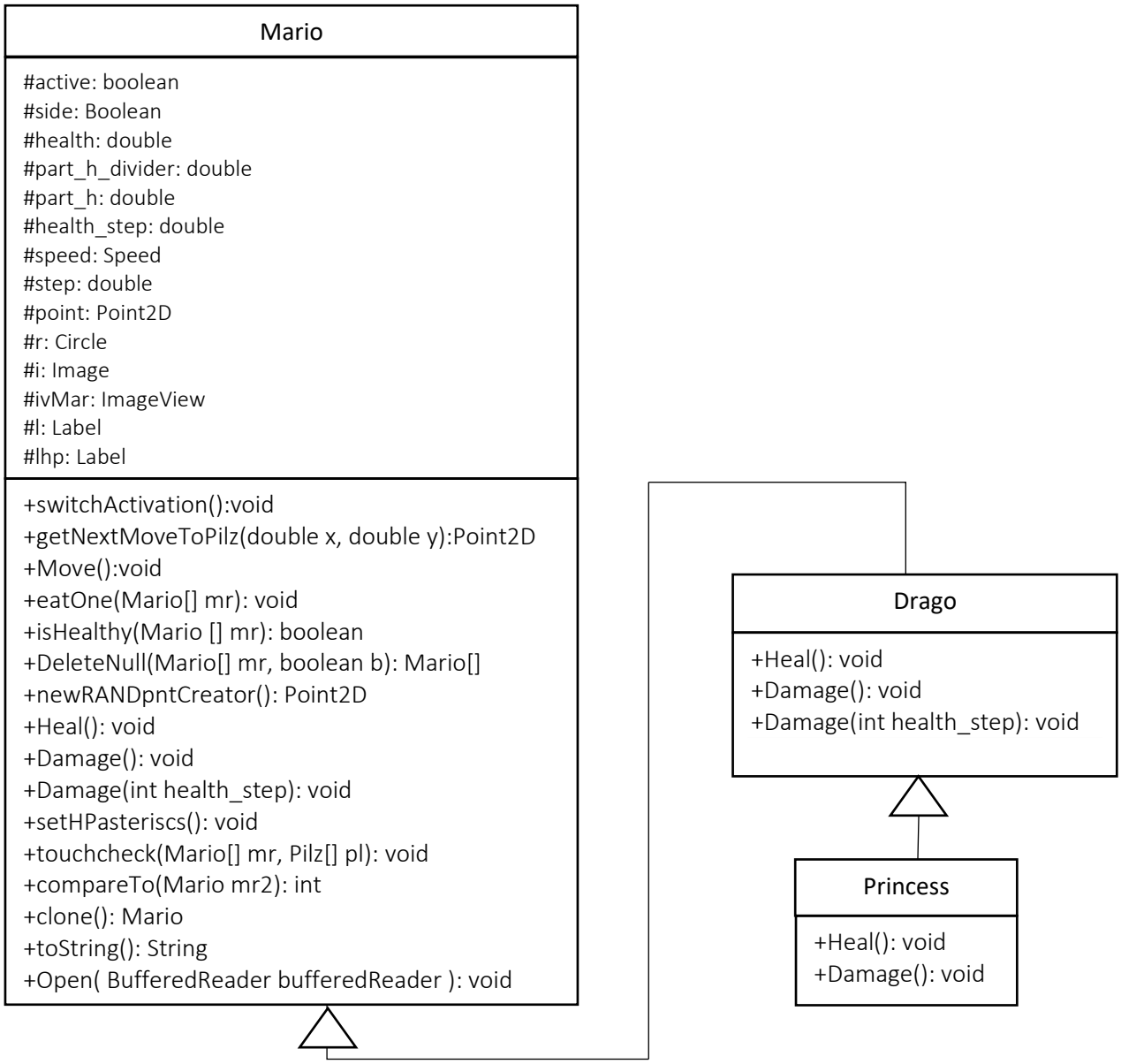


Рисунок 4.1 – Діаграма наслідування

Наслідування(див. рисунок 4.1) є однією з фундаментальних основ об’єктно-орієнтованого програмування, у якому клас “отримує” всі атрибути і операції класу, нащадком якого він є, і може додавати власні атрибути і операції.

Значення змінних в даній діаграмі наслідування:

- active – показник активності/неактивності мікрооб’єкта;
- side – сторона мікрооб’єкта (Добряк чи Поганець)
- health – здоров’я мікрооб’єкта
- part_h_divider: дільник здоров’я мікрооб’єкта
- part_h: частка здоров’я мікрооб’єкта
- health_step: крок зниження/підвищення здоров’я мікрооб’єкта
- speed: швидкість мікрооб’єкта
- step: крок мікрооб’єкта при “активованому” русі
- point: координати мікрооб’єкта
- r: круг – “аура” мікрооб’єкта
- i: зображення мікрооб’єкта
- ivMar: показ зображення мікрооб’єкта
- l: текст, індикатор сторони мікрооб’єкта(“G” або“B”)
- lhp: текст, візуальний індикатор здоров’я мікрооб’єкта

В основу даної ієрархії було покладено принцип “зростання за силою” , тобто чим далі мікрооб'єкт в ієрархії, тим він або витриваліший(Drago)(більше життєвих сил) або сильніший(Princess)(наносить більші пошкодження противникам).

4.1.2 Діаграми агрегації

Великий макрооб'єкт – Мапа (Wallpaper) можуть містити у собі деяку

кількість мікрооб’єктів (Mario, Drago, Princess), але при їх вилученні це не нашкодить самій Мапі. Таке відношення називається агрегацією (рисунок 4.2).

Рисунок 4.2 – Діаграма агрегації

4.1.3 Діаграми композиції

Композиції — це асоціації, які відповідають дуже сильній агрегації(рисунок 4.3). Це означає, що у композиціях ми також маємо справу з співвідношеннями ціле-частина, але тут зв’язок є настільки сильним, що частини не можуть існувати без цілого. Вони існують лише у межах цілого, після знищення цілого буде знищено і його частини.

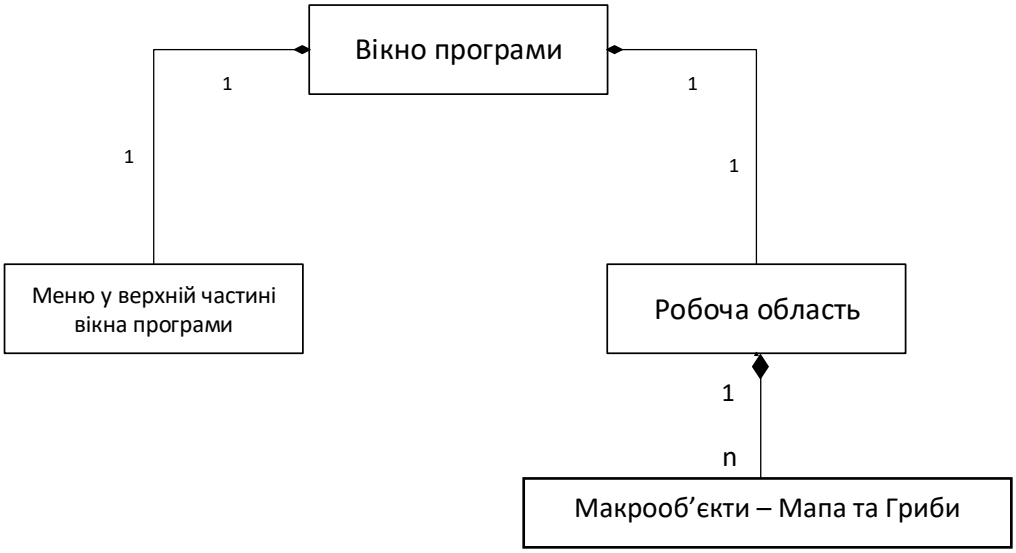


Рисунок 4.3 – Діаграма композиції

4.1.4 Діаграми асоціації

Асоціація—взаємна залежність між об’єктами різних класів, кожен з яких є рівноправним членом залежності (рисунок 4.4).

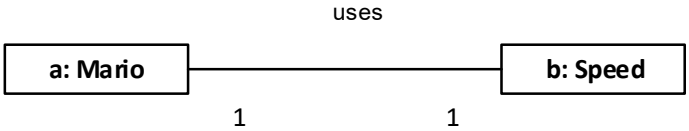
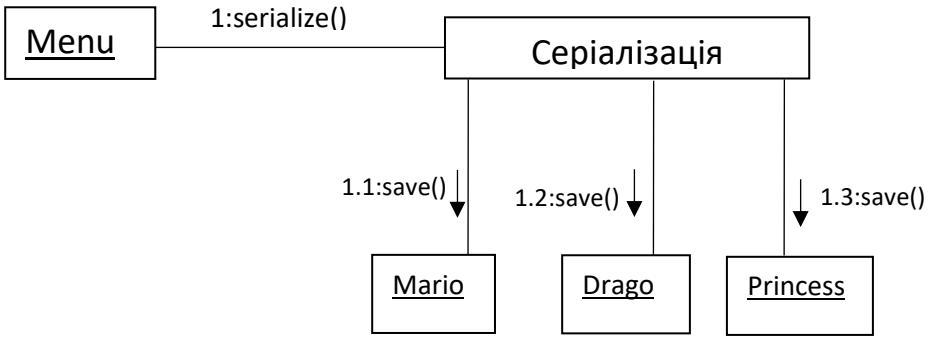


Рисунок 4.4 – Діаграма асоціації

Мікрооб’єкт Mario містить об’єкт класу Speed, який було додано для того, щоб глибинне копіювання стало необхідним.

4.2 Діаграми кооперації

Кооперація (collaboration) - специфікація безлічі об'єктів окремих класів, спільно взаємодіючих з метою реалізації окремих варіантів використання в загальному контексті модельованої системи. Її мета полягає в тому, щоб специфікувати особливості реалізації окремих варіантів використання або



найбільш значущих операцій в системі. Кооперація визначає структуру поведінки системи в термінах взаємодії учасників цієї кооперації (рисунок 4.5).

Рисунок 4.5 – Діаграми кооперації

4.3 Діаграми послідовності

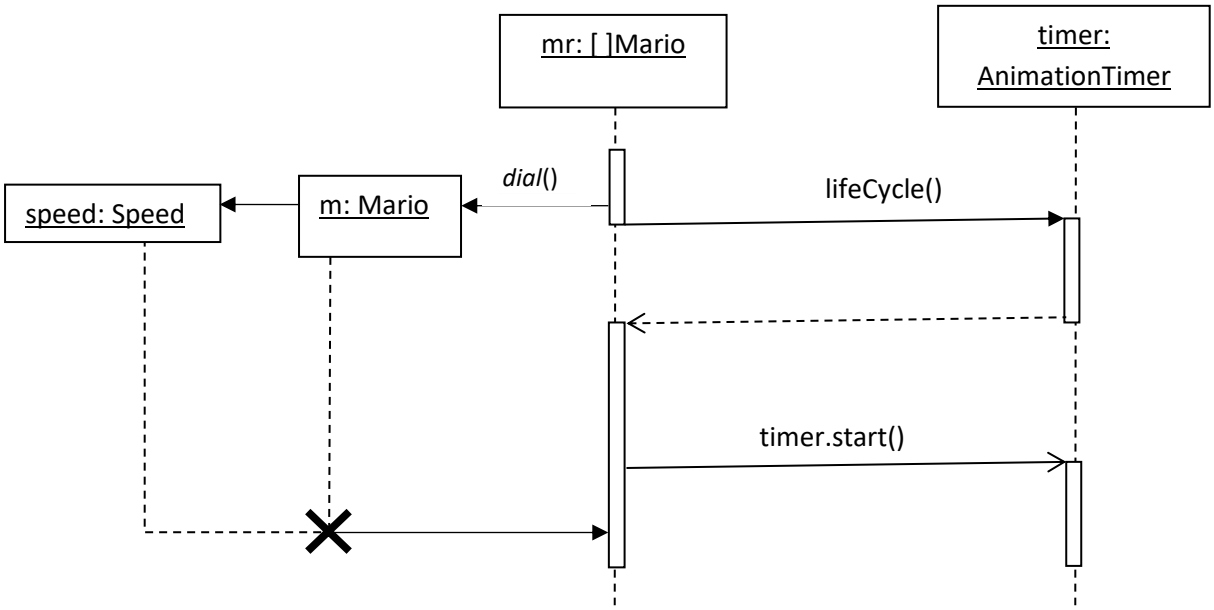


Рисунок 4.6 – Діаграма послідовності

Рисунок 4.6 демонструє життєвий цикл масиву мікрооб’єктів(lifeCycle() та start()) та створення за допомогою діалогового вікна dial() нового мікрооб’єкту і згодом його видалення.

4.4 Діаграми стану

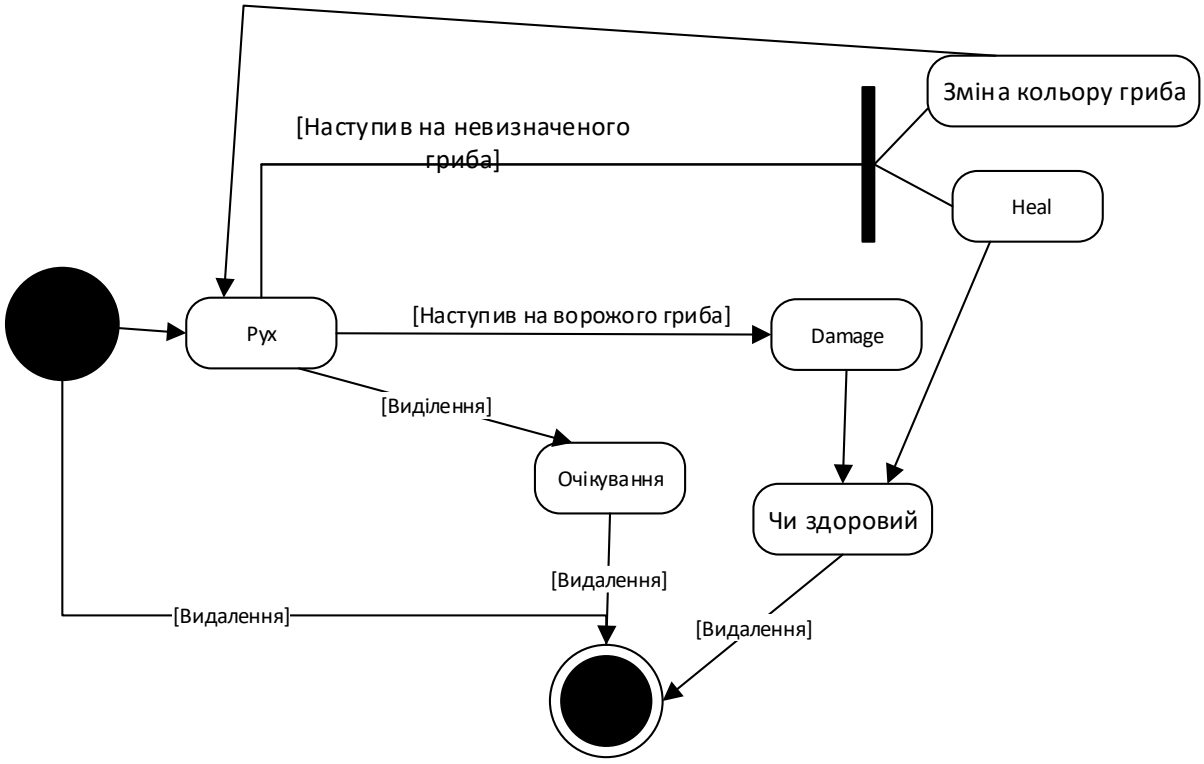


Рисунок 4.7 – Діаграма стану

Моделювання процесу функціонування складної системи, в реальному часі, зображено на рисунку 4.7 за допомогою діаграми стану.

5 ВИКОРИСТАННЯ ЗАСОБІВ ПРОГРАМУВАННЯ JAVAFX

JavaFX — платформа та набір інструментів для створення насичених інтернет-застосунків (англ. Rich Internet Applications, RIA) з можливістю підвантаження медіа та змісту [8]. JavaFX включає в себе набір утиліт, за допомогою яких веб-розробники та дизайнери можуть швидко створювати та надавати розвинуті інтернет-застосунки для десктопів, мобільних пристроїв, телебачення та інших платформ. JavaFX складається з JavaFX Script і JavaFX Mobile. Починаючи з випуску JavaFX 2.0 забезпечено можливість створення JavaFX-застосунків, написаних цілком мовою Java. Для розробки застосунків доступний багатий графічний і мультимедійний API, що спрощує створення візуальних програм.

В програмі було використано наступні засоби JavaFX:

- Клас JavaFX FileChooser (javafx.stage.FileChooser) - це діалогове вікно, за допомогою якого користувач може вибрати один або декілька файлів за допомогою провідника файлів з локального комп'ютера користувача. Показ діалогу JavaFX FileChooser здійснюється викликом методу showOpenDialog (). Файл, що повертається методом showOpenDialog (), - це файл, вибраний користувачем у FileChooser.
- MenuBar надає програмі JavaFX візуальне меню, подібне до того, що більшість настільних програм має у верхній частині вікна програми. Ви можете встановити графічну піктограму для меню за допомогою методу setGraphic (). Графічна піктограма відображатиметься поруч із текстовою позначкою меню. Екземпляр JavaFX Menu може запускати декілька подій, які можна прослуховувати у вашій програмі. Найпоширенішими подіями є: onShowing, onShown, onHiding, onHidden. Після створення екземпляра Menu необхідно додати до нього один або декілька екземплярів MenuItem. Кожен елемент MenuItem відповідає пункту меню в меню, до якого він додається.

- Клас Alert є підкласом класу Dialog і надає підтримку ряду попередньо побудованих типів діалогових вікон, які можуть бути легко показані користувачам для запиту відповіді. Тому для багатьох користувачів клас Alert є найбільш підходящим класом для їхніх потреб (на відміну від безпосереднього використання діалогового вікна). Альтернативно, користувачі, які хочуть запросити у користувача введення тексту або зробити вибір зі списку параметрів, краще обслуговуватимуться за допомогою TextInputDialog. і ChoiceDialog відповідно. Alert за замовчуванням - це вікно з модальністю Modelity.WINDOW_MODAL. Однак можна змінити за допомогою alert.initModality (Модальність).За замовчуванням будь-яке вікно, що відкриває діалогове вікно Alert, буде батьківським вікном вікна Alert. Існують такі різновиди Alert: Information Alert, Warning Alert, Error Alert, Confirmation Alert. Також в вікна Alert можна додавати свої особливі кнопки Custom Buttons.
- Клас AnimationTimer надає надзвичайно просту, але дуже корисну та гнучку функцію. Він дозволяє вказати метод, який буде викликатися в кожному кадрі. Використання цього метода не обмежене. Методи start () і stop () дозволяють запускати і зупиняти таймер.

6 РОЗРОБКА ПІДСИСТЕМИ СЕРІАЛІЗАЦІЇ/ДЕСЕРІАЛІЗАЦІЇ ДАНИХ

6.1 Розробка формату файлу

Параметр	Пояснення
1	Кількість Грибів
3302.0	Координата x зображення
302.0	Координата y зображення
3422.0	Координата x центру круга
411.0	Координата y центру круга
3392.0	Координата x тексту(Label)
362.0	Координата y тексту(Label)
0	Сторона Гриба
1	Порядковий номер Гриба
1	Кількість мікрооб’єктів
1	Номер класу(Mario – 1, Drago - 2)
true	Показник активності
true	Показник сторони
373.0	Координата x Групи мікрооб’єкта

108 3.0973	Координата у Групи мікрооб’єкта

Таблиця 6.1 – Формат файлу

У комп'ютерній науці в контексті зберігання даних серіалізація (serialization) є процесом перетворення структур даних або стану об'єкта у формат, який може зберігатися (наприклад, у файлі або буфері пам'яті) або передаватися (наприклад, через мережеве з'єднання) і відновлено пізніше (можливо, в іншому комп'ютерному середовищі) [9]. Коли результуючий ряд бітів перечитується відповідно до формату серіалізації, він може бути використаний для створення семантично ідентичного клону вихідного об'єкта. Для багатьох складних об'єктів, таких як ті, що широко використовують посилання, цей процес не є простим. Серіалізація об'єктно-орієнтованих об'єктів не включає жодного з пов'язаних з ними методів, з якими вони раніше були пов'язані.

Протилежна операція, вилучення структури даних з серії байтів, є десеріалізацією(deserealization).

У даній роботі використовується формат файлу зображений на таблиці 6.1.

6.2 Механізми введення виведення мови Java

Для роботи з файлами в даному випадку було використано такі засоби:

- Клас Java BufferedReader. Використовується для читання тексту з символьного вхідного потоку [10]. Він може використовуватися для читання рядків даних за методом readLine(). Рядок вважається закінченим, коли readLine() стикається з символом переходу на новий рядок ('\n'), або символом повернення каретки ('\r'). Цей клас пришвидшує роботу програми(з огляду на його мінімалістичність). Він успадковує клас Reader. Розмір буфера може бути заданий, або може бути використаний розмір за замовчуванням. Значення за замовчуванням досить велике для більшості цілей. Загалом, кожен запит на

читання, виконаний з пристрою читання, викликає відповідний запит на читання базового символу або потоку байтів.

- Клас Java FileWriter. Призначений для запису потоків символів. Конструктори цього класу припускають, що кодування символів і розмір байт-буфера за замовчуванням є прийнятними. Ці значення можна вказати, побудувавши OutputStreamWriter на файлі FileOutputStream.

7 КЕРІВНИЦТВО КОРИСТУВАЧА

7.1 Встановлення програми та необхідні файли

Для користувача доступно дві версі програми: архів файлів програмного коду або виконуваний файл.

Для першої версії достатньо скопіювати exe-файл необхідний для запуску.

Для другої версії потрібно мати встановлений архіватор, який працює з *.zip архівами, та IntelliJ IDEA 2019.1 (Community Edition).

Для роботи з програмою потрібно розархівувати папку, відкрити її в IntelliJ IDEA та запустити програму за допомогою кнопки Run. Після запуску програми користувач побачить вікно програми (рисунок 7.1)

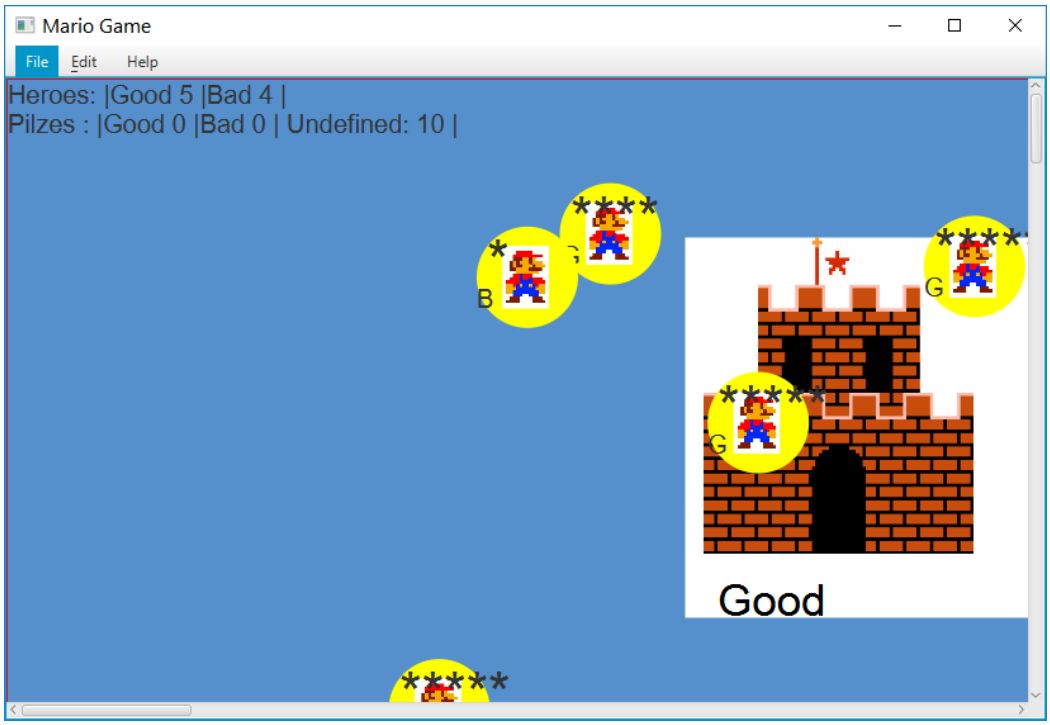


Рисунок 7.1 – Вікно програми

7.2 Функціональні клавіші

Для керування програмою використовуються наступні клавіші:

- Стрілки – використовуються для переміщення мікрооб’єктів

- ЛКМ – використовується для виділення мікрооб’єктів(колір круга змінюється з жовтого(невиділений) на червоний(виділений)) (рисунок 7.2)

Рисунок 7.2—Виділення мікрооб’єкта

- R – зміна з руху за замовчуванням(герої захоплюють найближчі до них гриби) на пасивний рух, тобто герої неохоче захоплюють гриби та швидкість героїв значно менша.

- B – зміна з руху за замовчуванням(герої захоплюють найближчі до них гриби) на рух до бази(рисунок 7.3), при цьому герої кожної зі сторін вертаються на базу, що належить їхній стороні. При цьому, якщо активувати клавішу R, то герої повністю припинять рух. Для відновлення руху за замовчуванням потрібно натиснути B та R ще раз.

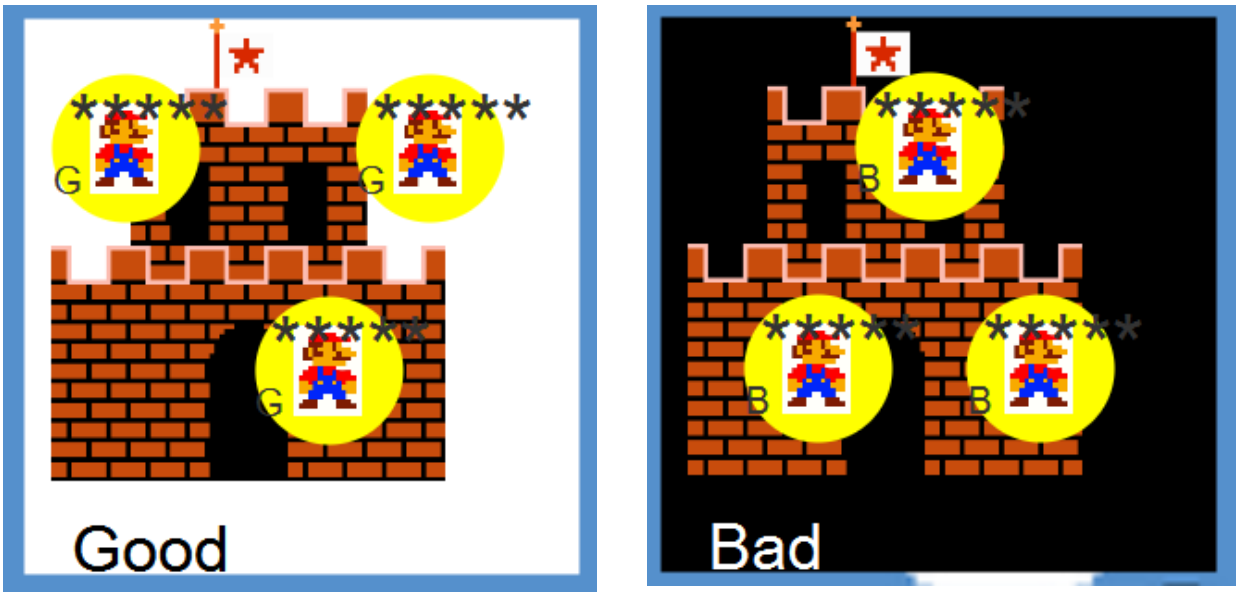


Рисунок 7.3 – Герої на своїх базах

- Insert – виклик діалогового вікна для додавання мікрооб’єкта (рисунок 7.4)

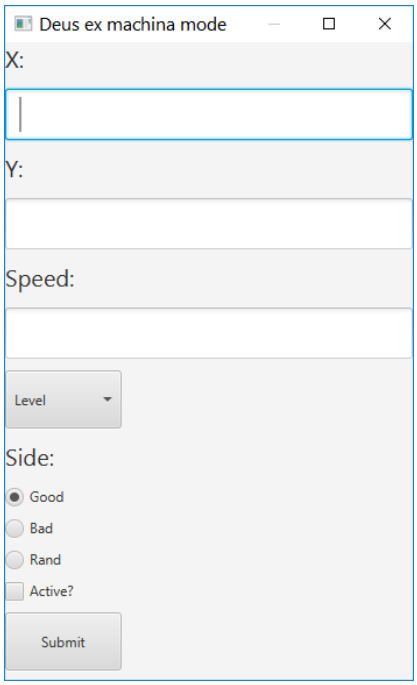
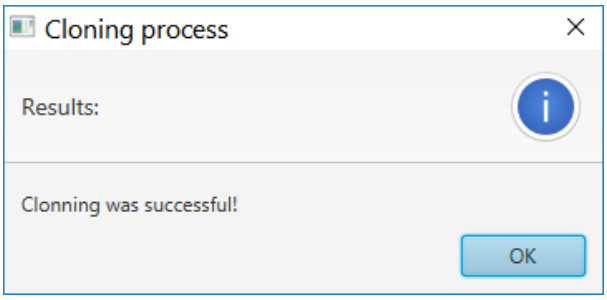


Рисунок 7.4—Виклик діалогового вікна

- Delete – видалення виділеного мікрооб’єкта(-ів)
- Escape – відмінити виділення всіх мікрооб’єктів

При виборі з випадаючого меню “Edit” ► “Clone” виділені макрооб’єкти



клонуються, про що нас сповіщає діалогове вікно (рисунок 7.5).

Рисунок 7.5 – Сповіщення про успішне клонування

ВИСНОВКИ

В процесі роботи над курсовою роботою було створено нову реалізацію програми по мотивах гри Super Mario, в якій було розкрито протистояння добра та зла, а також врирішено низку проблем: невибагливості до платформи, відкритості вихідного коду та відносно малого розміру програми.

Засобами обраної мови програмування (Java) було реалізовано методи, які забезпечують взаємодію мікрооб’єктів із елементами змодельованої території(меншими макрооб’єктами) та між собою.

Під час створення курсової роботи було отримано наступні навички: робота із візуальним середовищем програмування IntelliJ IDEA 2019.1 (Community Edition), класами та об’єктами, динамічними масивами, інтерфейсами, перевантаженням методів, серіалізацією/десеріалізацією, робота з засобами JavaFX такими як AnimationTimer, діалогові вікна, графічне відображення об’єктів.

Завдяки наявності відкритого коду, присутня можливість покращення програми користувачами з різних платформ завдяки платформонезалежності вихідного коду.

ПЕРЕЛІК ПОСИЛАНЬ

1. Super Mario. URL: https://en.wikipedia.org/wiki/Super_Mario
2. Рейтинг популярности языков программирования. URL: https://internetua.com/opublikovan-ocserednoi-reiting-populyarnosti-yazykov-programmirovaniya-1
3. Страуструп Б. Язык программирования C++ Специальное издание. Москва: Бином, 2011. 1136с.
4. Голубь Н. Особенности работы в языках C# / C++ с указателями и ссылками в объектно-ориентированном программировании. Vinnytsia, 22 - 25 May, 2018. Vinnytsia: VNTU, 2018. 260с.
5. Joshua B. Effective Java Programming Language Guide. Boston: Addison-Wesley Professional, 2018. 412p.
6. Jacobson I. Aspect-Oriented Software Development with Use Cases. Boston: Addison-Wesley Professional, 2004. 464p.
7. Бабенко Л. П. Інформаційна підтримка повторного використання у програмній інженерії на базі UML. Вінниця: кібернетика и системний аналіз, 2003. 82с.
8. Topley K. JavaFX™ developer's guide. Boston: Addison-Wesley Professional, 2011, 234p.
9. Бандура В.В. Java програмування. Структури, об'єкти, дані. Івано-Франківськ : ІФНТУНГ, 2018. 274с.
10. Сериализация в Java. URL: https://habr.com/ru/post/60317/

Д О Д А Т К И

Додаток А

Лістинг модуля Main.java

```
package sample;
import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.application.Platform;
import javafx.event.EventHandler;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.layout.BorderPane;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.stage.Stage;
import java.io.FileNotFoundException;
import java.util.Arrays;
import java.util.Random;
public class Main extends Application {
    public static Wallpaper wp;
    public static Pilz[] pl;
    public static BadBase badBase;
    public static GoodBase goodBase;
    public static Mario[] mr;
    public static Group groupMar = new Group();
    static AnimationTimer timer ;
    public static Random rnd = new Random(); // for spawning Pilzen
    static Group group = new Group();
    static Scene scene;
    static BorderPane layout;
    public static ScrollPane scrollPane;
    static Label mars = new Label();
    static Label pilzs = new Label();
    static  int t,f,u,b,g; // some counters
    public void SpawnWallpaper() throws FileNotFoundException {
        wp = new Wallpaper();
        group.getChildren().add(wp.getWallGrp());};
    static void SpawnPilz(int size) throws FileNotFoundException {
        pl = new Pilz[size];
        double px, py;                      // spawn Marss
        int i = 0;
        outer:
        while (i < pl.length) {
            int max = (int) Wallpaper.iv.getFitWidth();
            px = rnd.nextInt(max - 300);
            max = (int) Wallpaper.iv.getFitHeight();
            py = rnd.nextInt(max - 300);
            for (int j = 0; j < pl.length; j++) {
                if ((pl[j] != null) &&

pl[j].getGroup_Pilz().getBoundsInParent().intersects(px,py,300,300)
                          ||
Main.goodBase.g.getBoundsInParent().intersects(px, py, 300, 300)
                          ||
```

```
Main.badBase.g.getBoundsInParent().intersects(px, py, 300, 300)) {
                    continue outer;}} // goes to while and begins it once
again read about goto in Java }
            pl[i] = new Pilz(px, py); i++; }
        for (Pilz m : pl) { group.getChildren().add(m.g); } }
    public void SpawnMars (int size) throws FileNotFoundException {
        mr = new Mario[size];
        for (int i = 0; i < mr.length; i++){ mr[i] = new Mario(); }
        for (Mario m : mr) {groupMar.getChildren().add(m.getGroup()); } }
    public static void Show_teams(){  t=f=u=b=g=0;
        for(int i = 0; i<Main.mr.length; i++){
            if(Main.mr[i].getSide() == true) g++;
            if(Main.mr[i].getSide() == false) b++; }
        mars.setText("Heroes: |Good "+g+" |Bad "+b+" |");
        for(int i = 0; i<pl.length; i++){
            if(Main.pl[i].getSide() == 1) t++;
            if(Main.pl[i].getSide() == -1) f++;
            if(Main.pl[i].getSide() == 0) u++; }
        pilzs.setText("Pilzes : |Good "+t+" |Bad "+f+" | Undefined: "+u+"
|");
        pilzs.setFont(new Font("Arial", 20));
        mars.setFont(new Font("Arial", 20));
        if(u == 0){
            timer.stop();
            if(t>f) { Alert alert = new Alert(Alert.AlertType.INFORMATION);
                alert.setTitle("Congratulations!");
                alert.setHeaderText(null);
                alert.setContentText("The forces of Good had won
successfully!!!");
                Platform.runLater(alert::showAndWait);
                alert.setOnCloseRequest(e -> Platform.exit()); }
            if(t<f) {  Alert alert = new
Alert(Alert.AlertType.INFORMATION);
                alert.setTitle("Congratulations!");
                alert.setHeaderText(null);
                alert.setContentText("Alas, the forces of Bad had won...");
                Platform.runLater(alert::showAndWait);
                alert.setOnCloseRequest(e -> Platform.exit()); }
            if(t==f) { Alert alert = new
Alert(Alert.AlertType.INFORMATION);
                alert.setTitle("Congratulations!");
                alert.setHeaderText(null);
                alert.setContentText("Draw");
                Platform.runLater(alert::showAndWait);
                alert.setOnCloseRequest(e -> Platform.exit()); } }  }
    @Override
    public void start(Stage stage) throws Exception {
        SpawnWallpaper();
        badBase = new BadBase(Wallpaper.border.getWidth()-800, 100);
        group.getChildren().add(badBase.g);
        goodBase = new GoodBase(500, 100);
        group.getChildren().add(goodBase.g);
        SpawnPilz(10);
        SpawnMars(10);
        Group root = new Group(group, groupMar);
        scrollPane = new ScrollPane(root);

scrollPane.setMaxWidth(Wallpaper.border.getWidth());//Wallpaper.iv.getFitWi
```

```
dth()); 4600

scrollPane.setMaxHeight(Wallpaper.border.getHeight());//Wallpaper.iv.getFit
Width());
        scrollPane.setFitToHeight(true);
        scrollPane.setFitToWidth(true);
        layout = new BorderPane();
        TopMenu.SpawnMenu(); // adds it to the top of layout
        layout.setCenter(scrollPane);
        scene = new Scene(layout, 600,700);
        scene.setOnKeyPressed(new KeyPressedHandler());
        timer = new AnimationTimer() {
            @Override
            public void handle(long now) { lifeCycle(); }};
        t=f=u=b=g=0;
        for(int i = 0; i<Main.mr.length; i++){
            if(Main.mr[i].getSide() == true) g++;
            if(Main.mr[i].getSide() == false) b++; }
        for(int i = 0; i<pl.length; i++){
            if(Main.pl[i].getSide() == 1) t++;
            if(Main.pl[i].getSide() == -1) f++;
            if(Main.pl[i].getSide() == 0) u++; }
        Wallpaper.Tafel.getChildren().addAll(mars,pilzs);
        pilzs.relocate(0, 23);
        stage.setTitle("Hello World");
        stage.setScene(scene);
        timer.start();
        stage.show(); }
    public void lifeCycle(){
        boolean b = Mario.isHealthy(Main.mr);
        Main.mr = Mario.DeleteNull(Main.mr,b);        // Clean up dying
things
        Mario.eatOne(Main.mr);
        for(int i =0; i< Main.mr.length;i++) { mr[i].Move(); }
        Mario.touchcheck(Main.mr, Main.pl);
        Show_teams(); }
    private class KeyPressedHandler implements EventHandler<KeyEvent> {
        @Override
        public void handle(KeyEvent event) {
            if (event.getCode().equals(KeyCode.INSERT)) {
                Dial.dial();
                if (Dial.c == 0) ;
                else if (Dial.c == 1) {
                    try {Main.mr = Arrays.copyOf(Main.mr, Main.mr.length +
1);
                        Mario m = new Mario(Dial.active,
                                Dial.side,
                                Dial.x, Dial.y,
                                Dial.speed);
                        Main.mr[Main.mr.length - 1] = m;
                        groupMar.getChildren().add(Main.mr[Main.mr.length -
1].g);
                    } catch (FileNotFoundException e) {
e.printStackTrace(); } }
                else if (Dial.c == 3) {
                    try {Main.mr = Arrays.copyOf(Main.mr, Main.mr.length +
1);
                        Mario m = new Princess(Dial.active,
```

```
                                Dial.side,
                                Dial.x, Dial.y,
                                Dial.speed);
                            Main.mr[Main.mr.length - 1] = m;
                            group.getChildren().add(Main.mr[Main.mr.length -
1].g);
                        } catch (FileNotFoundException e) {
e.printStackTrace(); } }
                    else if (Dial.c == 2) {
                        try {Main.mr = Arrays.copyOf(Main.mr, Main.mr.length +
1);
                            Mario m = new Drago(Dial.active,
                                Dial.side,
                                Dial.x, Dial.y,
                                Dial.speed);
                            Main.mr[Main.mr.length - 1] = m;
                            group.getChildren().add(Main.mr[Main.mr.length -
1].g);
                        } catch (FileNotFoundException e) {
e.printStackTrace(); } }
                    Dial.active = false;Dial.c=0; }
                if (event.getCode().equals(KeyCode.DELETE)) {
                    int count=0;
                    for( Mario r :mr) { if(r.active) {
                        groupMar.getChildren().remove(r.g);
                        r.g.setVisible(false);
                        count++; } }
                    Mario [] tmp=new Mario[mr.length-count];
                    count=0;
                    for( Mario r :mr) { if(!r.active) { tmp[count]=r; count++;
} } mr= tmp; }
                if (event.getCode().equals(KeyCode.ESCAPE)) {
                    for(int i = 0; i< Main.mr.length; i++)
                    { if(Main.mr[i].active == true) {
                        Main.mr[i].active= !Main.mr[i].active;
                        if (Main.mr[i].active) Main.mr[i].r.setFill(Color.RED);
                        else Main.mr[i].r.setFill(Color.YELLOW); } } }
                if (event.getCode().equals(KeyCode.R)) {
Mario.SwitchRnd_Move(); }
                if (event.getCode().equals(KeyCode.B)) { Mario.SwitchTo_Base();
}
                if (event.getCode().equals(KeyCode.UP)) { for (Mario m : mr) {
m.Up(); } }
                if (event.getCode().equals(KeyCode.RIGHT)) { for (Mario m : mr)
{ m.Right(); } }
                if (event.getCode().equals(KeyCode.DOWN)) { for (Mario m : mr)
{ m.Down(); } }
                if (event.getCode().equals(KeyCode.LEFT)) { for (Mario m : mr)
{ m.Left(); } } } }
    public static void main(String[] args) { launch(args); } }
```

Додаток Б

Лістинг модуля Mario.java

```
package sample;
import javafx.beans.property.ReadOnlyObjectProperty;
import javafx.event.EventHandler;
import javafx.geometry.Bounds;
import javafx.geometry.Point2D;
import javafx.scene.Group;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import java.io.*;
import java.util.Comparator;
import java.util.Random;
public class Mario implements Cloneable{
    protected boolean active = false;
    public boolean isActive() {
        return active;
    }
    protected boolean side = true;
    public boolean getSide() {
        return side;
    }
    public void setSide(boolean side) { this.side = side;
        if (side == true) this.l.setText("G"); // Good
        else if (side == false) this.l.setText("B"); // Bad
        l.setFont(new Font("Arial", 20)); }
    protected static boolean to_Base = false;
    protected static boolean rnd_move = false;
    public static void SwitchTo_Base() {Mario.to_Base = !Mario.to_Base;}
    public static void SwitchRnd_Move() {Mario.rnd_move = !Mario.rnd_move;}
    protected double health = 500;
    protected static double max_health = 500;
    protected double part_h_divider = 5;
    protected double part_h = health/part_h_divider;
    protected double health_step = 1;
    static double newx;
    static double newy;
    protected Speed speed = new Speed();
    protected double step = 30;        // in methods Left Right Up Down
    static Mario[] tmp;
    static int n = 0;
    static Random random = new Random();
    protected Point2D point = newRANDpntCreator();
    protected Circle r;
    protected Image i;
    protected ImageView ivMar;
    protected Label l = new Label(), lhp = new Label();
    protected Group g;
    public Group getGroup() {
        return g;
    }
    Mario() throws FileNotFoundException {
        this.i = new Image(new FileInputStream("src/images/mario.png"));
        ImageView imageView = new ImageView(i);
```

```
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
        Circle aura = new Circle();
        aura.setRadius(40.0f);
        aura.setFill(Color.YELLOW);
        this.r = aura;
        setSide(random.nextBoolean());
        setHPasteriscs(); // label
        g = new Group(r, ivMar, lhp, l);
        ivMar.relocate(20, 15);
        lhp.relocate(8, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
     g.setLayoutX(Math.random() * 1000.0d);
        g.setLayoutY(Math.random() * 1000.0d);
     this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) { switchActivation(); } }); }
    Mario(boolean active, boolean side, double x, double y, double s) throws
FileNotFoundException {
        this.i = new Image(new FileInputStream("src/images/mario.png"));
        speed.setSpeed(s);
        ImageView imageView = new ImageView(i);
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
        Circle aura = new Circle();
        aura.setRadius(40.0f);
        this.active = active;
        aura.setFill(Color.YELLOW);
        this.r = aura;
        if (active) r.setFill(Color.RED);
        this.setSide(side);
        setHPasteriscs();
        g = new Group(r, ivMar, lhp, l); // relocating
        ivMar.relocate(20, 15);
        lhp.relocate(14, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
        g.setLayoutX(x);
        g.setLayoutY(y);
        this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) { switchActivation(); } }); }
    public void switchActivation() {
        this.active = !this.active;
        if (this.active)
            this.r.setFill(Color.RED);
        else
            this.r.setFill(Color.YELLOW); }
    public void setActive(boolean a) { this.active = a;
        if (this.active) this.r.setFill(Color.RED);
        else this.r.setFill(Color.YELLOW); }
    public void Left() { if (!active) return;
        double y = g.getLayoutX() - step;
        g.setLayoutX(y); }
    public void Up() { if (!active) return;
        double y = g.getLayoutY() - step;
        g.setLayoutY(y); }
    public void Right() { if (!active) return;
```

```
        double y = g.getLayoutX() + step;
        g.setLayoutX(y); }
    public void Down() { if (!active) return;
        double y = g.getLayoutY() + step;
        g.setLayoutY(y); }
    public static Point2D getNextMoveToPilz(double x, double y) {
        double[] distances = new double[Main.pl.length];
        for (int i = 0; i < Main.pl.length; i++) {
            if (Main.pl[i] != null) {
                distances[i] = Main.pl[i].getDistance(x, y);
            } else distances[i] = 1000000000.0; }
        double mind = 1000000000.0;
        int numd = 0;
        for (int i = 0; i < Main.pl.length; i++) {
            if ((Main.pl[i].getSide() == 0) && distances[i] < mind) {     //Magic
here
                mind = distances[i];
                numd = i; } }
        Point2D p2d = new Point2D(0, 0);
        if (Main.pl[numd] != null)
            p2d = new Point2D(Main.pl[numd].iv.getX(), Main.pl[numd].iv.getY());
        return p2d; }
    public void Move() {
        if (isActive()) return;
        if (to_Base == false && Mario.rnd_move == false) {
            Point2D p2d = getNextMoveToPilz(g.getLayoutX(), g.getLayoutY());
newx = p2d.getX();
            newy = p2d.getY();
            double dx = newx - g.getLayoutX();
            dx = (Math.abs(dx) > speed.getSpeed()) ? Math.signum(dx) *
speed.getSpeed() : dx;
            double dy = newy - g.getLayoutY();
            dy = (Math.abs(dy) > speed.getSpeed()) ? Math.signum(dy) *
speed.getSpeed() : dy;
g.setLayoutX(g.getLayoutX() + dx);
            g.setLayoutY(g.getLayoutY() + dy); }
        if (Mario.rnd_move == true && Mario.to_Base == false) {
            int x = (int) g.getLayoutX();
            int y = (int) g.getLayoutY();
            if (x == point.getX() && y == point.getY()) {
                point = newRANDpntCreator(); }
            int go = 1;
            if (x > point.getX()) { g.setLayoutX(g.getLayoutX() - go);
            } else if (y > point.getY()) {
                g.setLayoutY(g.getLayoutY() - go);
            } else if (x < point.getX()) {
                g.setLayoutX(g.getLayoutX() + go);
            } else if (y < point.getY()) {
                g.setLayoutY(g.getLayoutY() + go); } }
        if (Mario.to_Base == true && Mario.rnd_move == false) {
            if (this.getSide() == true) {
                newx = Main.goodBase.iv.getX()+Main.goodBase.r.getWidth()/2;
                newy = Main.goodBase.iv.getY()+Main.goodBase.r.getHeight()/2;
                double dx = newx - g.getLayoutX();
                dx = (Math.abs(dx) > speed.getSpeed()) ? Math.signum(dx) *
speed.getSpeed() : dx;
                double dy = newy - g.getLayoutY();
                dy = (Math.abs(dy) > speed.getSpeed()) ? Math.signum(dy) *
speed.getSpeed() : dy;
                g.setLayoutX(g.getLayoutX() + dx);
                g.setLayoutY(g.getLayoutY() + dy); }
            else { newx = Main.badBase.iv.getX()+Main.badBase.r.getWidth()/2;
                newy = Main.badBase.iv.getY()+Main.badBase.r.getHeight()/2;
```

```
double dx = newx - g.getLayoutX();
                dx = (Math.abs(dx) > speed.getSpeed()) ? Math.signum(dx) *
speed.getSpeed() : dx;
                double dy = newy - g.getLayoutY();
                dy = (Math.abs(dy) > speed.getSpeed()) ? Math.signum(dy) *
speed.getSpeed() : dy;
g.setLayoutX(g.getLayoutX() + dx);
                g.setLayoutY(g.getLayoutY() + dy); } } }
    public static void eatOne(Mario[] mr) {
        ReadOnlyObjectProperty<Bounds> iBounds;
        ReadOnlyObjectProperty<Bounds> jBounds;
        for (int i = 0; i < mr.length; i++) {
            for (int j = 0; j < mr.length; j++) {
                iBounds = mr[i].getGroup().boundsInParentProperty();
                jBounds = mr[j].getGroup().boundsInParentProperty();
                if (iBounds.get().intersects(jBounds.get())) {
                    if (mr[i].getSide() != mr[j].getSide()) {
                        mr[i].Damage(1);        // static polymorphism
                        mr[j].Damage(); } } } } }
    static int a = 0;
    public static boolean isHealthy(Mario [] mr) {
        Mario.a = 0;
        for (int i = 0; i < mr.length; i++) {
            if ( !(mr[i].health < 1)){
                Mario.a++; } }
        if(Mario.a < mr.length) return false;
        else return true; }
    public static Mario[] DeleteNull(Mario[] mr, boolean b) {
        Mario.n = 0;
        if(b) return mr;                        // if everyone is healthy
        for (int j = 0; j < mr.length; j++) {
            if (mr[j].health < 1.0) {
                Mario.n++;
                Main.groupMar.getChildren().remove(mr[j].getGroup());
                mr[j].g.setVisible(false); } }
        Mario.tmp = new Mario[mr.length - Mario.n];
        Mario.n = 0;
        for (int k = 0; k < mr.length; k++) {
            if ((mr[k].health < 1.0) == false) {
                tmp[n] = mr[k];
                ++Mario.n; } }
        return Mario.tmp; }
    public Point2D newRANDpntCreator() {
        Random rnd = new Random();
        int x = rnd.nextInt( (int)Main.wp.i.getWidth());
        int y = rnd.nextInt( (int)Main.wp.i.getHeight());
        Point2D point = new Point2D(x, y);
        return point; }
    public void Heal(){
        health_step = 1;
        health += health_step;// System.out.println("Heal!");
        setHPasteriscs(); }
    public void Damage(){
        health_step = 2;
        health -= health_step;//System.out.println("Damage!");
        setHPasteriscs(); }
    public void Damage(int health_step){
        health -= health_step;//System.out.println("Damage!");
        setHPasteriscs(); }
    public void setHPasteriscs() {
        lhp.setFont(new Font("Arial", 45));
        if (health == max_health) lhp.setText("*****");
        if (health <= 4 * part_h) lhp.setText("****");
```

```
        if (health <= 3 * part_h) lhp.setText("***");
        if (health <= 2 * part_h) lhp.setText("**");
        if (health <= part_h) lhp.setText("*"); }
    public static void touchcheck(Mario[] mr, Pilz[] pl) {
        for (int i = 0; i < mr.length; i++) {
            for (int j = 0; j < pl.length; j++) {
                if (mr[i].getGroup().getBoundsInParent().intersects
                        (pl[j].getGroup_Pilz().getBoundsInParent())
                        || pl[j].getGroup_Pilz().getBoundsInParent().contains
                        (mr[i].getGroup().getBoundsInParent())
                ) {
                    if(pl[j].getSide() != 0) {
                        if((mr[i].getSide() == true && pl[j].getSide() == 1))
mr[i].Heal();      // dynamic polimorphism here
                        else if (mr[i].getSide() == false && pl[j].getSide() == -
1) mr[i].Heal();
                        else mr[i].Damage(); }
                    else{ if (mr[i].getSide() == true) pl[j].setSide(1);
                        else                 pl[j].setSide(-1); } } }
            if (mr[i].getGroup().getBoundsInParent().intersects
                    (Main.goodBase.g.getBoundsInParent())) {
                if(mr[i].getSide() == true) {
                    mr[i].health = max_health;
                    mr[i].setHPasteriscs();}
                else mr[i].Damage(); }
            if(mr[i].getGroup().getBoundsInParent().intersects
                    (Main.badBase.g.getBoundsInParent())){
                if(mr[i].getSide() == false) {
                    mr[i].health = max_health;
                    mr[i].setHPasteriscs();}
                else mr[i].Damage(); }
            if (mr[i].getGroup().getBoundsInParent().getMinX() <=0 ||
                    mr[i].getGroup().getBoundsInParent().getMinY() <=0 ||
                    mr[i].getGroup().getBoundsInParent().getMinY() >=
Main.wp.border.getHeight()-70 ||
                    mr[i].getGroup().getBoundsInParent().getMinX() >=
Main.wp.border.getWidth()-70
            ){mr[i].active = false; mr[i].r.setFill(Color.YELLOW);} } }
    public int compareTo(Mario mr2) {
        if(this.speed.equals(mr2.speed)) {
            if( this.getGroup().getLayoutX()*this.getGroup().getLayoutY() >
                    mr2.getGroup().getLayoutX()*mr2.getGroup().getLayoutY())
return 1;
            if( this.getGroup().getLayoutX()*this.getGroup().getLayoutY() <
                    mr2.getGroup().getLayoutX()*mr2.getGroup().getLayoutY())
return -1;
            return 0; }
        else return this.speed.compareTo(mr2.speed); }
    public static Comparator<Mario> SpeedComparator
            = new Comparator<Mario>() {
        @Override
        public int compare(Mario o1, Mario o2) { return
o1.speed.compareTo(o2.speed); } };
    @Override
    public Mario clone() throws CloneNotSupportedException {
        this.switchActivation();
        Mario cloned = (Mario)super.clone();
        cloned.speed =  (Speed) this.speed.clone();
        return cloned; }
    @Override
    public String toString() {
        String side;
        if(getSide() == true) side = "Good";
```

```
        else side = "Bad";
        String clas = null;
        if((this.getClass().toString().equals("class sample.Mario"))){clas =
"Mario";}
        if((this.getClass().toString().equals("class sample.Drago"))){clas =
"Drago";}
        if((this.getClass().toString().equals("class sample.Princess"))){clas =
"Princess";}
        String rez= "\n[ "+clas+", "+side+", " +
                Integer.toString((int)getGroup().getLayoutX())+", " +
                Integer.toString((int)getGroup().getLayoutY())+", " +
                Integer.toString((int)speed.getSpeed())+" ]";
        return rez; }
    public void Open( BufferedReader bufferedReader ) throws IOException {
        String text;
        text = bufferedReader.readLine();
        boolean act= (text.equals("true"))?true:false;
        setActive(act);
        text = bufferedReader.readLine();
        act= (text.equals("true"))?true:false;
        setSide(act);
        text = bufferedReader.readLine();
        this.getGroup().setLayoutX(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.getGroup().setLayoutY(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.speed.setSpeed(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.health = (Double.parseDouble(text));
        setHPasteriscs(); }
    public void Save( FileWriter fileWriter ) throws IOException {
        fileWriter.write(getKlas()); // first level = Mario
        fileWriter.write("\n");
        fileWriter.write( Boolean.toString(isActive()));
        fileWriter.write("\n");
        fileWriter.write( Boolean.toString(getSide()));
        fileWriter.write("\n");
        fileWriter.write( Double.toString(getGroup().getLayoutX()) );
        fileWriter.write("\n");
        fileWriter.write( Double.toString(getGroup().getLayoutY()) );
        fileWriter.write("\n");
        fileWriter.write( Double.toString(speed.getSpeed()));
        fileWriter.write("\n");
        fileWriter.write( Double.toString(this.health) );
        fileWriter.write("\n"); }
    public String getKlas(){
        String clas = null;
        if((this.getClass().toString().equals("class sample.Mario"))){clas = "1";}
        if((this.getClass().toString().equals("class sample.Drago"))){clas = "2";}
        if((this.getClass().toString().equals("class sample.Princess"))){clas =
"3";}
        return clas; } }
```

Додаток В

Лістинг модуля Princess.java

```
     package sample;
import javafx.event.EventHandler;
import javafx.scene.Group;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
public class Princess extends Drago {
    Princess(boolean active,boolean side,double x, double y, double
s) throws FileNotFoundException {
        super(active, side, x, y, s);
        this.i = new Image(new
FileInputStream("src/images/princess.png"));
speed = new Speed();
        speed.setSpeed(s);
ImageView imageView = new ImageView(i);
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
Circle aura = new Circle();
        aura.setRadius(40.0f);
        this.active = active;
        aura.setFill(Color.YELLOW);
        this.r = aura;
if (active) r.setFill(Color.RED);
this.setSide(side);
        setHPasteriscs();
        g = new Group(r, ivMar, lhp, l);
        ivMar.relocate(20, 15);
        lhp.relocate(14, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
        g.setLayoutX(x);
        g.setLayoutY(y);
        this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
@Override public void handle(MouseEvent event) { switchActivation();
} }); }
     Princess() throws FileNotFoundException {
         super();
         this.i = new Image(new
FileInputStream("src/images/princess.png"));
        ImageView imageView = new ImageView(i);
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
```

```
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
        Circle aura = new Circle();
        aura.setRadius(40.0f);
        aura.setFill(Color.YELLOW);
        this.r = aura;
        setSide(random.nextBoolean());
        setHPasteriscs(); // label
        g = new Group(r, ivMar, lhp, l);
        ivMar.relocate(20, 15);
        lhp.relocate(8, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
        g.setLayoutX(Math.random() * 1000.0d);
        g.setLayoutY(Math.random() * 1000.0d);
        this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) {
switchActivation(); } }); }
    @Override
     public void Heal(){
        health_step = 4;
        health += health_step;// System.out.println("Heal!");
        setHPasteriscs(); }
    @Override
    public void Damage(){
        health_step = 1;
        health -= health_step;//System.out.println("Damage!");
        setHPasteriscs(); } }
```

Додаток Г

Лістинг модуля Speed.java

```
     package sample;
import javafx.event.EventHandler;
import javafx.scene.Group;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
public class Princess extends Drago {
    Princess(boolean active,boolean side,double x, double y, double
s) throws FileNotFoundException {
        super(active, side, x, y, s);
        this.i = new Image(new
FileInputStream("src/images/princess.png"));
speed = new Speed();
        speed.setSpeed(s);
ImageView imageView = new ImageView(i);
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
Circle aura = new Circle();
        aura.setRadius(40.0f);
        this.active = active;
        aura.setFill(Color.YELLOW);
        this.r = aura;
if (active) r.setFill(Color.RED);
this.setSide(side);
        setHPasteriscs();
        g = new Group(r, ivMar, lhp, l);
        ivMar.relocate(20, 15);
        lhp.relocate(14, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
        g.setLayoutX(x);
        g.setLayoutY(y);
        this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
@Override public void handle(MouseEvent event) { switchActivation();
} }); }
     Princess() throws FileNotFoundException {
         super();
         this.i = new Image(new
FileInputStream("src/images/princess.png"));
        ImageView imageView = new ImageView(i);
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
```

```
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
        Circle aura = new Circle();
        aura.setRadius(40.0f);
        aura.setFill(Color.YELLOW);
        this.r = aura;
        setSide(random.nextBoolean());
        setHPasteriscs(); // label
        g = new Group(r, ivMar, lhp, l);
        ivMar.relocate(20, 15);
        lhp.relocate(8, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
        g.setLayoutX(Math.random() * 1000.0d);
        g.setLayoutY(Math.random() * 1000.0d);
        this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) {
switchActivation(); } }); }
    @Override
     public void Heal(){
        health_step = 4;
        health += health_step;// System.out.println("Heal!");
        setHPasteriscs(); }
    @Override
    public void Damage(){
        health_step = 1;
        health -= health_step;//System.out.println("Damage!");
        setHPasteriscs(); } }
```

Додаток Д

Лістинг модуля Pilz.java

```
      package sample; import javafx.scene.Group; import
javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import java.io.*;
public class Pilz {
   private int side = 0; // Undefined
    public int getSide() {
        return side; }
    public void setSide(int a){ if (a<=(-1)) { this.side = -1;
this.r.setFill(Color.BLACK); }
        if (a>= 1) { this.side = 1;    // 1 Good
            this.r.setFill(Color.WHITE); }
        else if(a == 0) { this.side = a;
            this.r.setFill(Color.YELLOW); } }
    public  Circle r;
    Image i;
    ImageView iv;
    Label l;
    protected Group g;
    double damage;
    static int counter = 0;
    int count=0;
    Pilz(double x, double y) throws FileNotFoundException {
        i = new Image(new FileInputStream("src/images/Pilz.png"));
        ImageView imageView = new ImageView(i);
        imageView.setX(50+x);
        imageView.setY(25+y);
        imageView.setFitHeight(250);
        imageView.setFitWidth(270);
        imageView.setPreserveRatio(true); //Setting the preserve ratio of the
image view
        this.iv = imageView;
        Circle aura = new Circle();
        aura.setCenterX(170+x);
        aura.setCenterY(150+y);
        aura.setRadius(150.0f);
        aura.setFill(Color.YELLOW);
        r = aura;
        Label num = new Label();
        num.setText(Integer.toString(counter));   counter++; count = counter;
        num.setFont(new Font("Arial", 20));
        num.setTranslateX(140+x);
        num.setTranslateY(100+y);
        this.l = num;
        this.g = new Group(r, this.iv, this.l ); }
Pilz() throws FileNotFoundException { double x,y; x=y=0;
        i = new Image(new FileInputStream("src/images/Pilz.png"));
        ImageView imageView = new ImageView(i);
        imageView.setX(50+x);
        imageView.setY(25+y);
        imageView.setFitHeight(250);
        imageView.setFitWidth(270);
        imageView.setPreserveRatio(true); //Setting the preserve ratio of the
image view
```

```
        this.iv = imageView;
        Circle aura = new Circle();
        aura.setCenterX(170+x);
        aura.setCenterY(150+y);
        aura.setRadius(150.0f);
        aura.setFill(Color.YELLOW);
        r = aura;
        Label num = new Label();
        num.setText(Integer.toString(counter));  counter++; count =counter;
        num.setFont(new Font("Arial", 20));
        num.setTranslateX(140+x);
        num.setTranslateY(100+y);
        this.l = num;
        this.g = new Group(r, this.iv, this.l ); }
public Group getGroup_Pilz() { return g; }
    public double getDistance( double x, double y )  {
        double rezult = (iv.getX() - x)* (iv.getX() - x)+ (iv.getY() - y)*
(iv.getY() - y) ;
        return rezult; }
public void Open( BufferedReader bufferedReader ) throws IOException {
        String text;
        text = bufferedReader.readLine();
        this.iv.setX(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.iv.setY(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.r.setCenterX(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.r.setCenterY(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.l.setTranslateX(Double.parseDouble(text));
        text = bufferedReader.readLine();
        this.l.setTranslateY(Double.parseDouble(text));
        text = bufferedReader.readLine();
        setSide(Integer.parseInt(text));
        text = bufferedReader.readLine();
        this.count = Integer.parseInt(text); }
   public void Save( FileWriter fileWriter ) throws IOException {
        fileWriter.write( Double.toString(iv.getX()) );
        fileWriter.write("\n");
        fileWriter.write( Double.toString(iv.getY()) );
        fileWriter.write("\n");
        fileWriter.write( Double.toString(r.getCenterX()) );
        fileWriter.write("\n");
        fileWriter.write( Double.toString(r.getCenterY()) );
        fileWriter.write("\n");
        fileWriter.write( Double.toString(l.getTranslateX()) );
        fileWriter.write("\n");
        fileWriter.write( Double.toString(l.getTranslateY()) );
        fileWriter.write("\n");
        fileWriter.write( Integer.toString(getSide()) );
        fileWriter.write("\n");
        fileWriter.write( Integer.toString(this.count) );
        fileWriter.write("\n"); } }
```

Додаток Е

Лістинг модуля Wallpaper.java

```
     package sample;
import javafx.scene.Group;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
public class Wallpaper {
    Image i;
    public  static ImageView iv;
    Label l;
    private static Group subi;
    public static Rectangle border;
    static Group Tafel = new Group();
    static Group g;
    Wallpaper() throws FileNotFoundException {
        i = new Image(new
FileInputStream("src/images/Mwallpaper.png"));
        ImageView imageView = new ImageView(i);
        imageView.setFitWidth(4600);
        imageView.setFitHeight(4000);
        this.iv = imageView;
        border = new Rectangle(0,0,this.iv.getFitWidth(),
this.iv.getFitHeight());
        border.setFill(Color.TRANSPARENT);
        border.setStroke(Color.RED);
        subi = new Group(this.iv,border);
        Group sub = new Group( subi, Tafel);
        this.g = sub; }
    public static Group getWallGrp() { return g; } }
```

Додаток Ж

Лістинг модуля TopMenu.java

```
     package sample;
import javafx.application.Platform;
import javafx.scene.control.*;
import javafx.stage.FileChooser;
import javafx.stage.Stage;
import java.io.*;
import java.util.Arrays;
import java.util.logging.Level;
import java.util.logging.Logger;
public class TopMenu {
    public static Stage stage;
    public static void SpawnMenu(){
        Menu fileMenu = new Menu("File");
        MenuItem openFile = new MenuItem("Open...");
        openFile.setOnAction(e -> deserialize());
        fileMenu.getItems().add(openFile);
        MenuItem saveFile = new MenuItem("Save...");
        saveFile.setOnAction(e -> serialize());
        fileMenu.getItems().add(saveFile);
        fileMenu.getItems().add(new SeparatorMenuItem());
        fileMenu.getItems().add(new MenuItem("Settings..."));
        fileMenu.getItems().add(new SeparatorMenuItem());
        MenuItem exitFile = new MenuItem("Exit...");
        exitFile.setOnAction(e -> Platform.exit()); // fileMenu.hide()
        fileMenu.getItems().add(exitFile);
        Menu editMenu = new Menu("_Edit");
        MenuItem cloneFile = new MenuItem("Clone...");
        cloneFile.setOnAction(e -> {
            try { cloneMar(); } catch (CloneNotSupportedException ex) {
ex.printStackTrace(); } });
        editMenu.getItems().add(cloneFile);
        MenuItem info = new MenuItem("Info...");
        info.setOnAction(e -> info());
        editMenu.getItems().add(info);
        Menu helpMenu = new Menu("Help");
        CheckMenuItem showLines = new CheckMenuItem("Show Line Numbers");
        showLines.setOnAction(e -> { if (showLines.isSelected())
System.out.println("Program will now display line numbers");
            else System.out.println("Hiding line number"); });
        CheckMenuItem autoSave = new CheckMenuItem("Enable Autosave");
        autoSave.setSelected(true);
        helpMenu.getItems().addAll(showLines, autoSave);
        MenuBar menuBar = new MenuBar();
        menuBar.getMenus().addAll(fileMenu,editMenu,helpMenu);
        Main.layout.setTop(menuBar); }

    private static void deserialize() {
        String currentDir= System.getProperty("user.dir");
        File initDirectory = new File(currentDir);
        FileChooser fileChooser = new FileChooser();
        FileChooser.ExtensionFilter extFilter =
                new FileChooser.ExtensionFilter("TXT files (*.txt)",
"*.txt");
```

```
        fileChooser.getExtensionFilters().add(extFilter);
        fileChooser.setInitialDirectory(
                initDirectory);
        File file = fileChooser.showOpenDialog(stage);
        if(file != null) {
            BufferedReader bufferedReader = null;
            try { bufferedReader = new BufferedReader(new
FileReader(file));
                String text;
                text = bufferedReader.readLine();
                int hmany = Integer.parseInt(text);
                for (Pilz r : Main.pl)
{Main.group.getChildren().remove(r.g); }
                Pilz.counter =0;
                Main.pl = new Pilz[hmany];
                for (int i = 0; i < hmany; i++) {
                    Main.pl[i] = new Pilz();
                    Main.pl[i].Open(bufferedReader);
                    Main.group.getChildren().add(Main.pl[i].g); }
                text = bufferedReader.readLine();
                hmany = Integer.parseInt(text);
                for (Mario r : Main.mr) {
                    Main.groupMar.getChildren().remove(r.getGroup()); }
                Main.mr = new Mario[hmany];
                int type;
                for (int i = 0; i < hmany; i++) {
                    text = bufferedReader.readLine();
                    type = Integer.parseInt(text);
                    if (type == 1) {
                        Main.mr[i] = new Mario();
                        Main.mr[i].Open(bufferedReader);
                    } else if (type == 2) {
                        Main.mr[i] = new Drago();
                        Main.mr[i].Open(bufferedReader);
                    } else if (type == 3) {
                        Main.mr[i] = new Princess();
                        Main.mr[i].Open(bufferedReader); }
                    Main.groupMar.getChildren().add(Main.mr[i].getGroup());
}
            } catch (FileNotFoundException ex) {
                Logger.getLogger(Main.class.getName()).log(Level.SEVERE,
null, ex);
            } catch (IOException ex) {
                Logger.getLogger(Main.class.getName()).log(Level.SEVERE,
null, ex);
            } finally { try { bufferedReader.close(); } catch (IOException
ex) {

Logger.getLogger(Main.class.getName()).log(Level.SEVERE, null, ex); } } } }
    private static void serialize() {
        String currentDir = System.getProperty("user.dir");
        File initDirectory = new File(currentDir);
        FileChooser fileChooser = new FileChooser();
        FileChooser.ExtensionFilter extFilter =
                new FileChooser.ExtensionFilter("TXT files (*.txt)",
"*.txt");
        fileChooser.getExtensionFilters().add(extFilter);
        fileChooser.setInitialDirectory(
```

```
                initDirectory);
        File file = fileChooser.showSaveDialog(stage);
        if(file != null) {
            try { FileWriter fileWriter = null;
                fileWriter = new FileWriter(file);
fileWriter.write(Integer.toString(Main.pl.length));
                fileWriter.write("\n");
                for( Pilz r:Main.pl ) { r.Save(fileWriter); }
fileWriter.write(Integer.toString(Main.mr.length));
                fileWriter.write("\n");
for( Mario r:Main.mr )
                { r.Save(fileWriter); }
                fileWriter.close();
            } catch (IOException ex) {
Logger.getLogger(Main.class.getName()).log(Level.SEVERE, null, ex); } } }
    public static void cloneMar() throws CloneNotSupportedException {
        Main.timer.stop();
        Mario[] tmparray; // = java.util.Arrays.copyOf(Main.mr,
Main.mr.length);
        Mario[] array = Main.mr;
        for (int i = 0; i < Main.mr.length; i++) {
            if (Main.mr[i].isActive()) {
                tmparray = java.util.Arrays.copyOf(
                        Main.mr, Main.mr.length + 1);
                tmparray[Main.mr.length] = Main.mr[i].clone();
                Main.mr = tmparray;
                System.out.println("Cloned, " +
                        "but it is impossiblle to clone() the graphical
part."); } }
        if(array.length < Main.mr.length) {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);
            alert.setTitle("Cloning process");
            alert.setHeaderText("Results:");
            alert.setContentText("Clonning was successful!");
            alert.showAndWait(); } Main.timer.start(); }
    static String genStat() {
        System.out.println("[General info]");
        String s = Arrays.toString(Main.mr);            //???
        return s; }
    private static void info() {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("Info");
        alert.setHeaderText("Here's arrrray:");
        alert.setContentText(genStat());
        alert.showAndWait(); } }
```

Додаток И

Лістинг модуля BadBase.java

```
    package sample;
import javafx.scene.Group;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
public class BadBase { Image i; ImageView iv; Rectangle r;
Group g; Label l;
BadBase(double x, double y) throws FileNotFoundException {
i = new Image(new
FileInputStream("src/images/castle_base.png"));
        ImageView imageView = new ImageView(i);
        imageView.setX(50+x);
        imageView.setY(25+y);
        imageView.setFitHeight(250);
        imageView.setFitWidth(270);
        imageView.setPreserveRatio(true); //Setting the
preserve ratio of the image view
        this.iv = imageView;
        this.l = new Label("Bad");
        this.l.setFont(new Font("Arial", 35));
        this.l.setTextFill(Color.WHITE);
        this.l.setTranslateX(60+x);
        this.l.setTranslateY(290+y);
        this.r = new Rectangle(x+35, y+25, 300, 300);
        this.r.setFill(Color.BLACK);
        this.g = new Group();
        this.g.getChildren().addAll(this.r, this.iv,
this.l); } }
```

Додаток К

Лістинг модуля GoodBase.java

```
    package sample;
import javafx.scene.Group;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
public class GoodBase { Image i; ImageView iv; Rectangle r;
Group g; Label l;
    GoodBase(double x, double y) throws
FileNotFoundException {
        i = new Image(new
FileInputStream("src/images/castle_base.png"));
        ImageView imageView = new ImageView(i);
        imageView.setX(50+x);
        imageView.setY(25+y);
        imageView.setFitHeight(250);
        imageView.setFitWidth(270);
        imageView.setPreserveRatio(true); //Setting the
preserve ratio of the image view
        this.iv = imageView;
        this.l = new Label("Good");
        this.l.setFont(new Font("Arial", 35));
        this.l.setTextFill(Color.BLACK);
        this.l.setTranslateX(60+x);
        this.l.setTranslateY(290+y);
        this.r = new Rectangle(x+35, y+25, 300, 300);
        this.r.setFill(Color.WHITE);
        this.g = new Group();
        this.g.getChildren().addAll(this.r, this.iv,
this.l); } }
```

Додаток Л

Лістинг модуля Dial.java

```
     package sample;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.sceKKne.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.VBox;
import javafx.scene.shape.Circle;
import javafx.scene.text.Font;
import javafx.stage.Modality;
import javafx.stage.Stage;
import java.util.Random;
public class Dial {
    static boolean answer = false;
    static boolean side ;
    static boolean active = false;
    public static double x = 0;
    public static double y = 0;
    static Circle r;
    static Image i;
    static ImageView iv;
    static Label l;
    static double speed;
    static int c;
    public static boolean dial() {
        Stage window = new Stage();
        window.initModality(Modality.APPLICATION_MODAL);//Block
events to other windows
        window.setTitle("Deus ex machina mode");
        Label label1 = new Label("X:");
        label1.setFont(new Font(20));
        TextField textField1 = new TextField();
        textField1.setFont(new Font(20));
        Label label2 = new Label("Y:");
        label2.setFont(new Font(20));
        TextField textField2 = new TextField();
        textField2.setFont(new Font(20));
        Label label_speed = new Label("Speed:");
        label_speed.setFont(new Font(20));
        TextField textField_speed = new TextField();
        textField_speed.setFont(new Font(20));
        ToggleGroup group = new ToggleGroup();
        Label label = new Label("Side:");
        label.setFont(new Font(20));
        RadioButton good = new RadioButton("Good");
        good.setToggleGroup(group);
        RadioButton bad = new RadioButton("Bad");
```

```
        bad.setToggleGroup(group);
        RadioButton rand = new RadioButton("Rand");
        rand.setToggleGroup(group);
        good.setSelected(true);
        CheckBox ctrl = new CheckBox("Active?");
        ComboBox comboBox = new ComboBox<>();
        comboBox.getItems().addAll( "1. Mario", "2. Yoshi", "3.
Princess" );
        comboBox.setPromptText("Level");
        Button submit = new Button("Submit");
        submit.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                answer = true;
                x = Integer.parseInt(textField1.getText());
                y = Integer.parseInt(textField2.getText());
                speed =
Double.parseDouble(textField_speed.getText());
                if (good.isSelected()) {
                    side = true; }
                if (bad.isSelected()) {
                    side = false; }
                if (rand.isSelected()) {
                    Random random = new Random();
                    side = random.nextBoolean(); }
                if (ctrl.isSelected()) { active = true; }
                if (comboBox.getValue() == "1. Mario") c=1;
                if (comboBox.getValue() == "2. Yoshi")c=2;
                if (comboBox.getValue() =="3. Princess")c=3;
                window.close(); } });
        comboBox.setPrefSize(100,50);
        submit.setPrefSize(100,50);
        VBox layout = new VBox(10);
        layout.getChildren().addAll( label1, textField1, label2,
textField2, label_speed, textField_speed, comboBox, label, good,
bad, rand, ctrl, submit );
        Scene scene = new Scene(layout, 400, 600);
        window.setScene(scene);
        window.showAndWait();
        return answer; } }
```

Додаток М

Лістинг модуля Drago.java

```
     package sample;
import javafx.event.EventHandler;
import javafx.scene.Group;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
public class Drago extends Mario {
    Drago( boolean active, boolean side, double x, double y, double s )
throws FileNotFoundException {
        super(active, side, x, y, s);
        this.i = new Image(new FileInputStream("src/images/dino.png"));
        speed = new Speed();
        speed.setSpeed(s);
        ImageView imageView = new ImageView(i);
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
        Circle aura = new Circle();
        aura.setRadius(40.0f);
        this.active = active;
        aura.setFill(Color.YELLOW);
        this.r = aura;
        if (active) r.setFill(Color.RED);
        this.setSide(side);
        setHPasteriscs();
        g = new Group(r, ivMar, lhp, l);
        ivMar.relocate(20, 15);
        lhp.relocate(14, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
        g.setLayoutX(x);
        g.setLayoutY(y);
        this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) { switchActivation(); }
}); }
     Drago() throws FileNotFoundException {
         super();
         this.i = new Image(new FileInputStream("src/images/dino.png"));
        ImageView imageView = new ImageView(i);
        imageView.setFitHeight(50);
        imageView.setFitWidth(50);
        imageView.setPreserveRatio(true);
        this.ivMar = imageView;
        Circle aura = new Circle();
        aura.setRadius(40.0f);
        aura.setFill(Color.YELLOW);
        this.r = aura;
```

```
        setSide(random.nextBoolean());
        setHPasteriscs(); // label
        g = new Group(r, ivMar, lhp, l);
        ivMar.relocate(20, 15);
        lhp.relocate(8, 0);
        l.relocate(0, 45);
        r.relocate(0, 0);
        g.setLayoutX(Math.random() * 1000.0d);
        g.setLayoutY(Math.random() * 1000.0d);
        this.g.setOnMouseClicked(new EventHandler<MouseEvent>() {
            @Override
            public void handle(MouseEvent event) { switchActivation(); }
}); }
  @Override
     public void Heal(){
        health_step = 2;
        health += health_step;// System.out.println("Heal!");
        setHPasteriscs(); }
    public void Damage(){
        health_step = 1;
        health -= health_step;//System.out.println("Damage!");
        setHPasteriscs(); }
        @Override
    public void Damage(int h){
        health_step = h;
        health -= health_step;//System.out.println("Damage!");
        setHPasteriscs(); } }
```